AKADEMIKA

BEL-DIT

DIGITAL IC TRAINER

EXPERIMENTAL MANUAL

…. A MESSAGE FROM

Today's system designers are faced with tomorrow's problems. BASIC ELECTRONICS is one of the important subject need to teach while learning electronics.
It is our vision to provide you with the product you need for training ensuring lasting reliability & quality.

OUR MOTTO;

- Light years ahead – refers to leadership.

As leaders in our industry in India, We are totally committed to servicing as the standard against which all are measured in the areas of

• Design • Quality • Value • Delivery • Support.

We are truly light years ahead of our competition in this area. That means that you our valued customers are guaranteed satisfaction.

As you will move through this manual you will quickly discover that we have a complete, truly innovative & superior training products we are so committed to quality that we back our products with a complete comprehensive warranty.

AKADEMIKA LAB SOLUTIONS

Sr. No. 15/8/1, Unit No.9, Kruti Industrial Estate, Opp.
Karishma Society, Off Karve Road, Kothrud, Pune,
Maharashtra- 411038
INDIA
Contact No.:+91-7447438443
Email: lab@akademika.in
www.akademika.in

SAFETY RULES

- Read carefully and follow the instructions mentioned in this manual. This user manual includes all the important points about the installation, use and the maintenance of the product. Keep this manual always with you, for quick reference.

- After unpacking the product, arrange all the accessories in proper order, so that their integrity is checked with the packing list. Also, ensure that the accessories have no visible damage.

- Before connecting the power supply to the kit, be sure that the jumpers and the connecting chords are connected correctly, as per the experiment.

- This kit must be employed only for the use for which it has been conceived, i.e. as educational kit and must be used under the direct survey of expert personnel. Any other use is inadvisable and dangerous too. The manufacturer cannot be considered responsible for eventual damages due to improper, wrong or unreasonable uses.

- In case of any fault or malfunctioning in the trainer kit, turn off the power supply. Please do not tamper the kit. If you require our service, kindly contact the service centre for technical assistance.

- The kits are liable to malfunction/under-perform if they are not operated under standard environmental conditions of temperature and humidity.

WARRANTY

This kit is warranted against defects in workmanship and materials. Any failure due to defect in either workmanship or material should occur under normal use within a year from the original date of purchase, such failure will be corrected free of charge to the purchaser by repair or replacement of defective part or parts. When the failure is result of user's neglect, natural disaster or accident, we would charge for repairs, regardless of the warranty period. The warranty does not cover include perishable items like connecting chords, crystals, etc. and other imported items.

Conditions and Limitations

The warranty is void and inapplicable if the defective product is not brought or sent to our authorized service center or sales outlet within the warranty period. Defective product will be Akademika Lab Solutions sole judgment. The defective product will be replaced with a new one or repaired, without charge or with charge.

In the warranty period if the service is needed, the purchaser should get in touch with the service center or the sales outlet. The purchaser should return the product to the service center or the sales outlet at his or her sole expense. The loss and damage in transit will be outside the preview of this warranty. A returned product must be accompanied by a written description of the defects. Type and Model No. of the kit has to be mentioned specifically. We return the product to the purchaser at our expense. In case the warranty does not cover the product on Akademika Lab Solutions judgment, we would repair the product after obtaining prior permission from purchaser who would receive an estimate statement about the repairing charges. In such cases, Akademika Lab Solutions bares the transporting expenses required to send back all the repaired products for the moment, and then repairs and transporting expenses will be charged against the purchaser by the statement of accounts.

When the authorized sales agents sell our products, they must notify the purchaser of the warranty contents, but they have no rights to stretch the meaning of original warranty contents or to offer an additional warranty. Akademika Lab Solutions does not provide any other promise or suggestive warranty and hold no liability for the damage caused by negligence, abnormal use or natural disaster. Akademika Lab Solutions is not responsible for the damages even if it is notified of above dangers in advance as well.

For more special service or overall repairs, maintenance and up gradation of products, be sure to contact our service center or the sales outlet.

INDEX

TECHNICAL SPECIFICATIONS

- Basic logic gate IC's NOT (7404), OR (7432), AND (7408), NOR (7402),NAND (7400), EX-0R (7486)
- NAND and NOR gates as Universal Logic Gates
- De-Morgan's theorem I and II
- Boolean Equation
- Half Adder, Full Adder, Half Subtractor, Full Subtractor
- Basic Flip-Flops RS (using NOR), JK (7476), D (7474), MS-JK (7476), D (7474) and T (using JK)
- Ripple Counter (7490)
- Synchronous Binary Counter (74191)
- 4 Bit Ring Counter using 7476
- Decade / BCD counter using 7490
- Universal Shift register 74194
- 9 bit parity Generator / Checker (74280)
- Multiplexer (74153) and De-multiplexer (74138)
- BCD to seven segment decoder (7447)
- 4 bit Comparator (7485)
- Binary to Gray, Gray to Binary, Binary to BCD, BCD to Binary, BCD to Excess 3, Excess 3 to BCD
- 16 switches to provide Logic 0 & 1 inputs with LED indication
- 16 LED's to observe the output logic states
- 20 pin ZIF socket

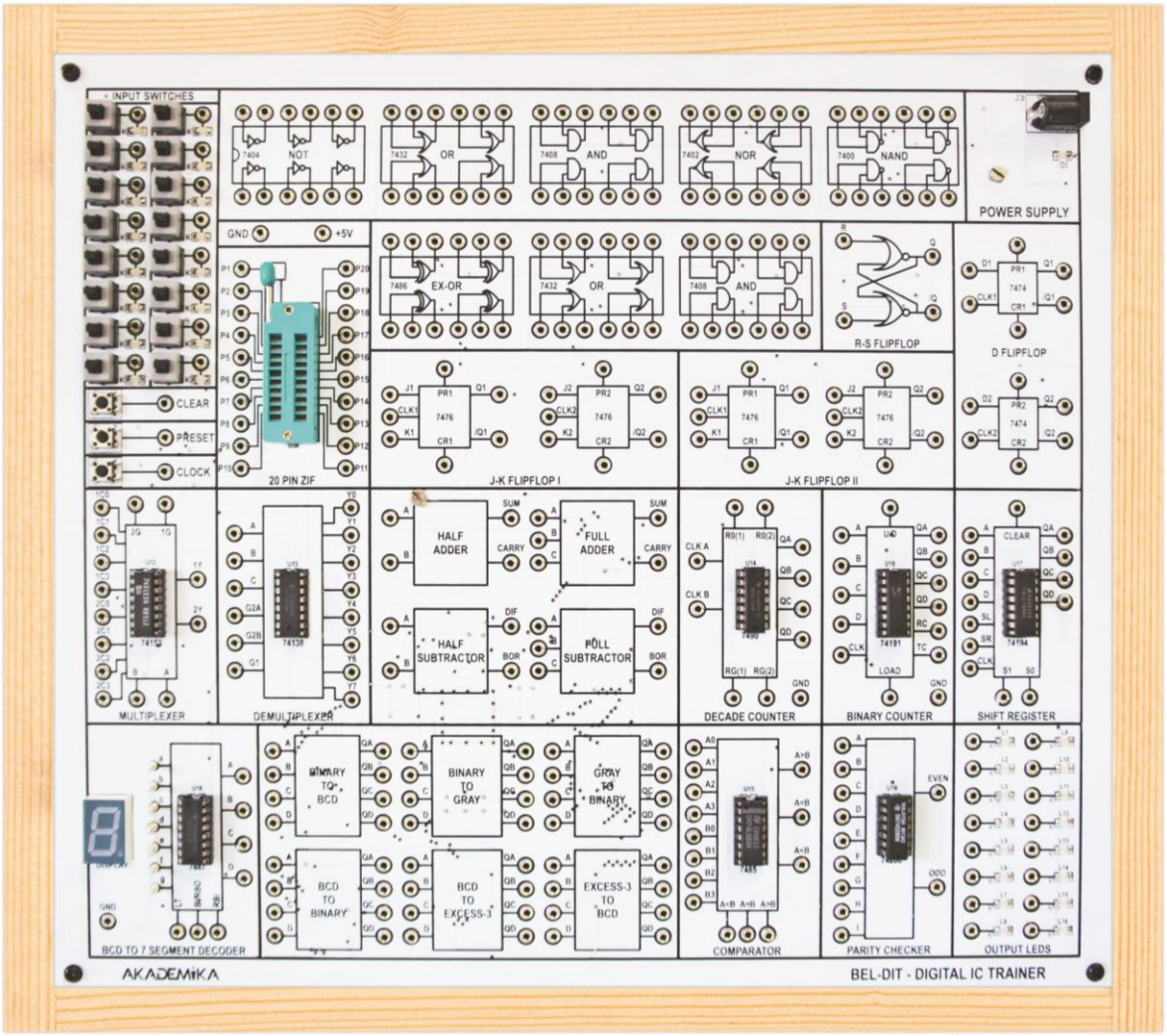
Manual clock to observe the counter operation

Accessories

	BEL-DIT	Quantity
1	Short Links PCS2 Patch cord Male To Male	15
2	Long Links PCS2 Patch cord Male To Male	5
3	EXPT Manual	1
4	0.47µF/63v electro cap	2
5	10K Resistor MFR	2
6	7.5V/350ma Adaptor	1
7	500K_3296 Preset	1

Note: Reading Tables and Graphs provided in this manual are taken on sample trainer board hence actual readings and graphs may differ from the readings and graphs shown in this manual. Readings and graphs in this manual are to be treated as sample only for reference purpose to get idea of characteristics of the respective component.

FUNCTIONAL BLOCK



INTRODUCTION

The **Digital IC Trainer** is a very versatile and descriptive digital laboratory instrument with built-in regulated power supply. The trainer covers topics on all basic logic gates, universal gates, flip-flops, counters, registers, multiplexer and de-multiplexer, seven segment display driver, parity generator / checker and code converters.

Easy interconnections between circuits allow designing various logic functions using on board resources. It has 16 logic switches for providing inputs to digital ICs and 16 LED indicators to check the outputs from the digital ICs. The system comes with descriptive experimental manual.

EXPERIMENT NO. 1

EXPERIMENT NO.1

NAME:-

Study of Logic Gates

OBJECTIVE:-

To study about logic gates and verify their truth tables.

EQUIPMENTS:-

- BEL-DIT Board
- Power supply

THEORY:-

Circuit that takes the logical decision and the process are called logic gates. Each gate has one or more input and only one output. OR, AND and NOT are basic gates. NAND, NOR and X-OR are known as universal gates. Basic gates form these gates.

AND Gate
The AND gate performs a logical multiplication commonly known as AND function. The output is high when both the inputs are high. The output is low level when any one of the inputs is low.

OR Gate
The OR gate performs a logical addition commonly known as OR function. The output is high when any one of the inputs is high. The output is low level when both the inputs are low.

NOT Gate
The NOT gate is called an inverter. The output is high when the input is low. The output is low when the input is high.

NAND Gate
The NAND gate is a contraction of AND-NOT. The output is high when both inputs are low and any one of the input is low .The output is low level when both inputs are high.

NOR Gate
The NOR gate is a contraction of OR-NOT. The output is high when both inputs are low. The output is low when one or both inputs are high.

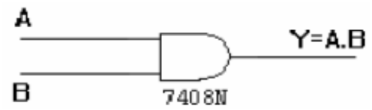
X-OR Gate
The output is high when any one of the inputs is high. The output is low when both the inputs are low and both the inputs are high.

PROCEDURE:-

- Do the Connections as per symbol and indent marked on PCB.
- Give the logic input to the gate under test as per symbol from INPUT
- SWITCHES section.
- Connect output of gate under test to any of the led from OUTPUT LED section
- Observe the output on LEDS from OUTPUT SECTION and verify the truth table.

AND Gate

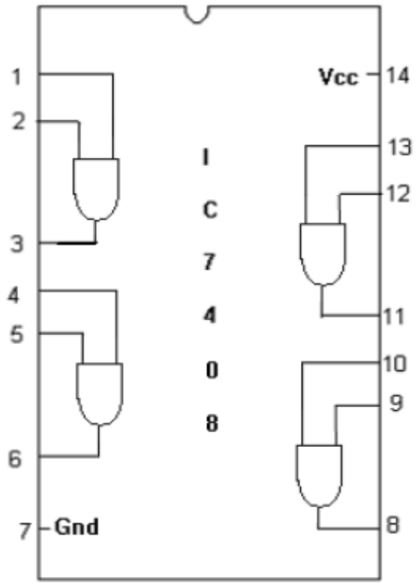
SYMBOL:



TRUTH TABLE

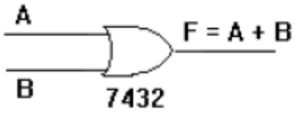
A	B	A.B
0	0	0
0	1	0
1	0	0
1	1	1

PIN DIAGRAM:

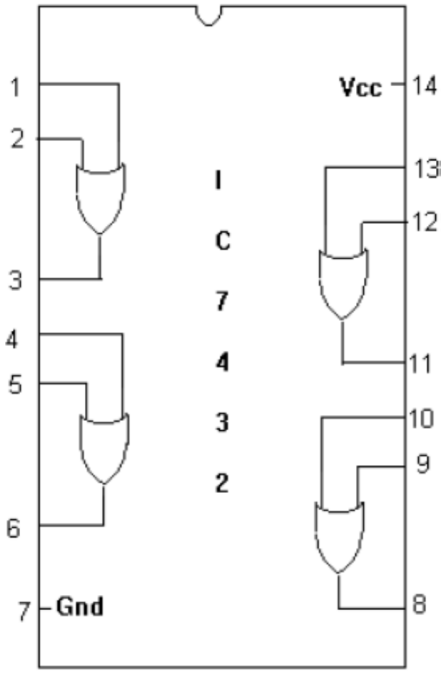


OR Gate

SYMBOL :



PIN DIAGRAM :

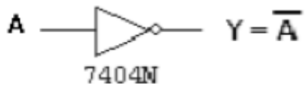


TRUTH TABLE

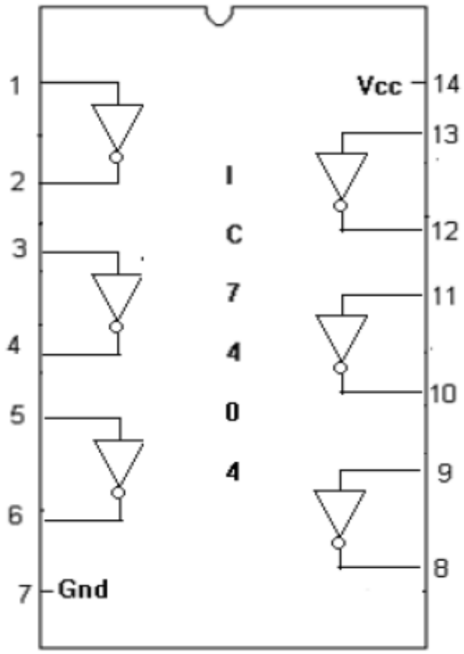
A	B	A+B
0	0	0
0	1	1
1	0	1
1	1	1

NOT Gate

SYMBOL:



PIN DIAGRAM:

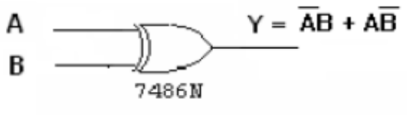


TRUTH TABLE :

A	$\overline{A}$
0	1
1	0

XOR Gate

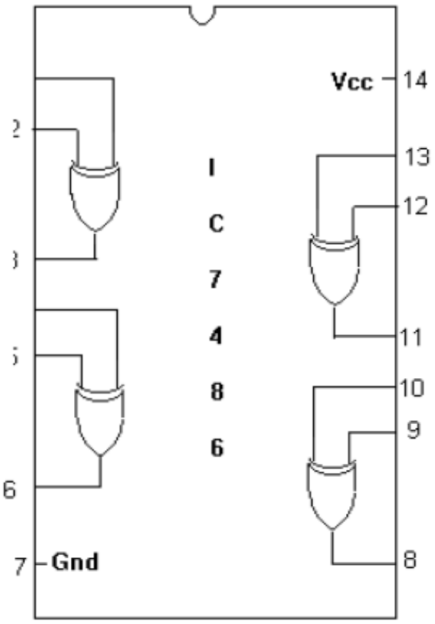
SYMBOL :



TRUTH TABLE :

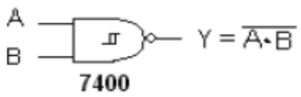
A	B	$\overline{A}B + A\overline{B}$
0	0	0
0	1	1
1	0	1
1	1	0

PIN DIAGRAM :



NAND Gate

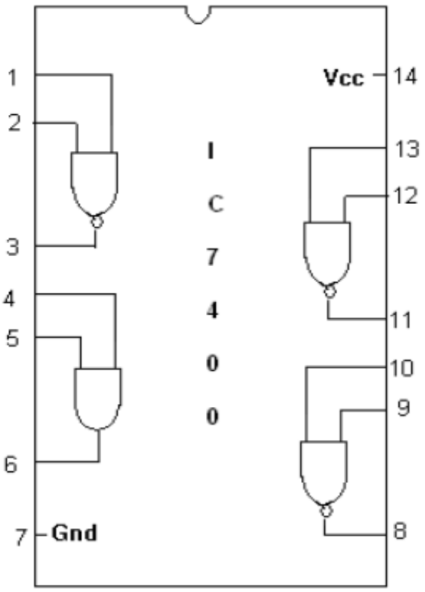
SYMBOL:



TRUTH TABLE

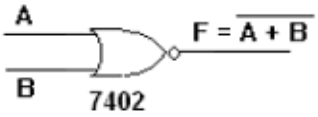
A	B	$\overline{A \cdot B}$
0	0	1
0	1	1
1	0	1
1	1	0

PIN DIAGRAM:

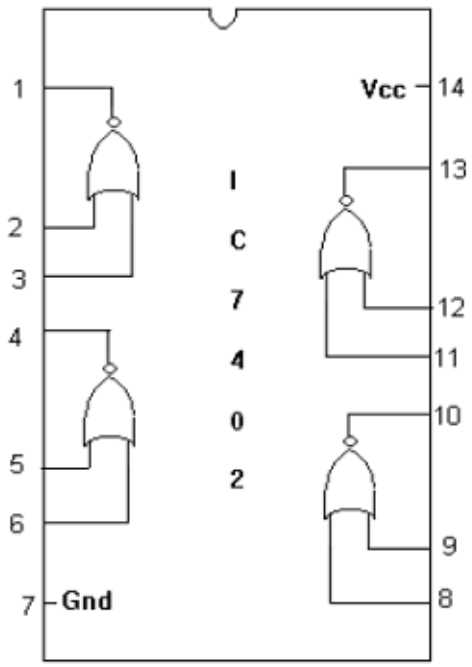


NOR Gate

SYMBOL :



PIN DIAGRAM :



TRUTH TABLE

A	B	$\overline{A+B}$
0	0	1
0	1	0
1	0	0
1	1	0

CONCLUSION:-

The truth tables for various logic gates like AND, OR, NAND, NOT, EX-OR, NOR are verified

EXPERIMENT NO. 2

EXPERIMENT NO.2

NAME:-

Study of Universal Gates

OBJECTIVE:-

To study NAND and NOR gates as Universal Logic Gates.

EQUIPMENTS:-

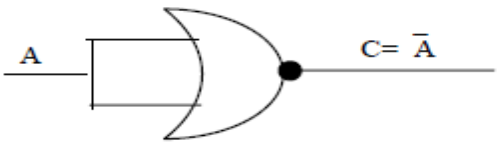
- BEL-DIT Board
- Power supply

PROCEDURE:-

- Do the connections as per circuit shown below
- Apply logic inputs from INPUT SWITCHES section to the input of the NOR gate.
- Connect output of gate under test to any of the led from OUTPUT LED section
- Verify the truth table as per schematic and Gate which is constructed.

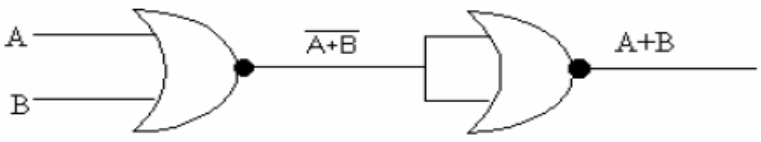
1) Realization of All Basic Gates Using NOR Gate

NOT GATE

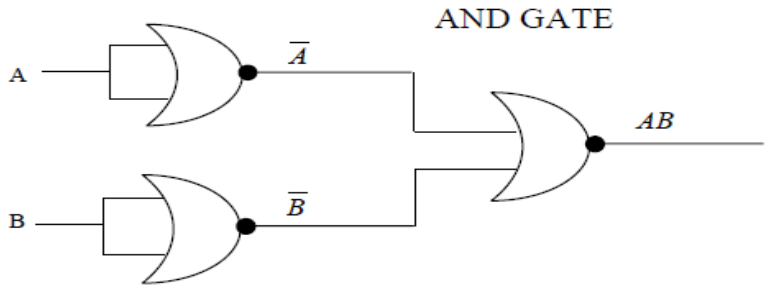


A	C=Ā
0	1
1	0

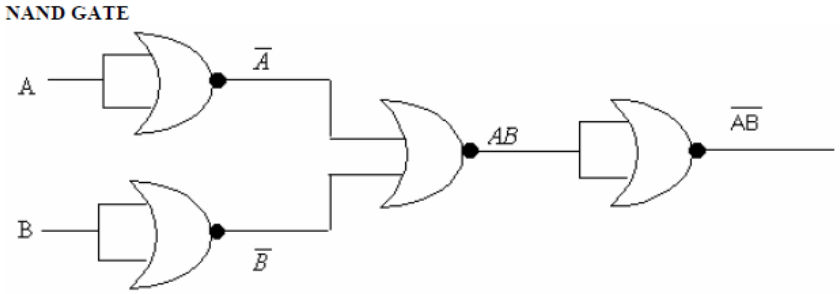
OR GATE



A	B	A+B
0	0	0
0	1	1
1	0	1
1	1	1



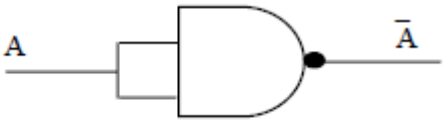
A	B	AB
0	0	0
0	1	0
1	0	0
1	1	1



A	B	$\overline{AB}$
0	0	1
0	1	1
1	0	1
1	1	0

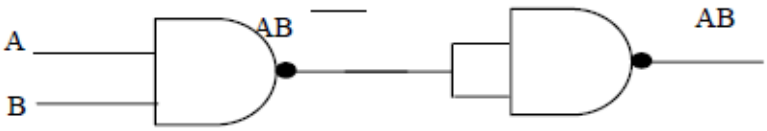
2) Realization of All Basic Gates Using NAND Gate

NOT GATE:



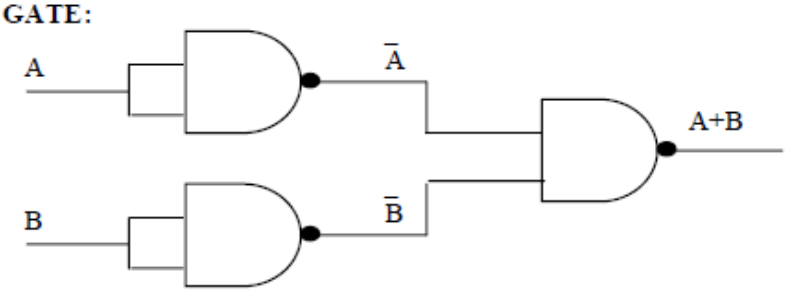
A	C=Ā
0	1
1	0

AND GATE:



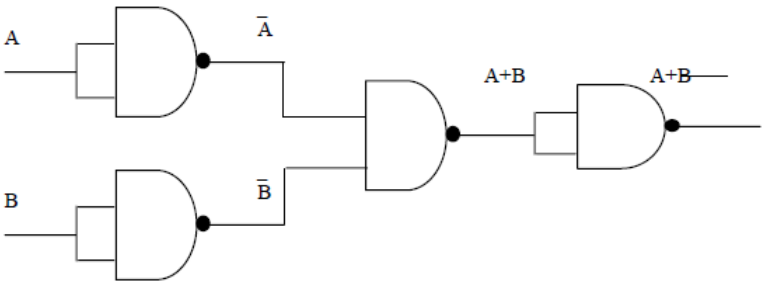
A	B	AB
0	0	0
0	1	0
1	0	0
1	1	1

OR GATE:



A	B	A+B
0	0	0
0	1	1
1	0	1
1	1	1

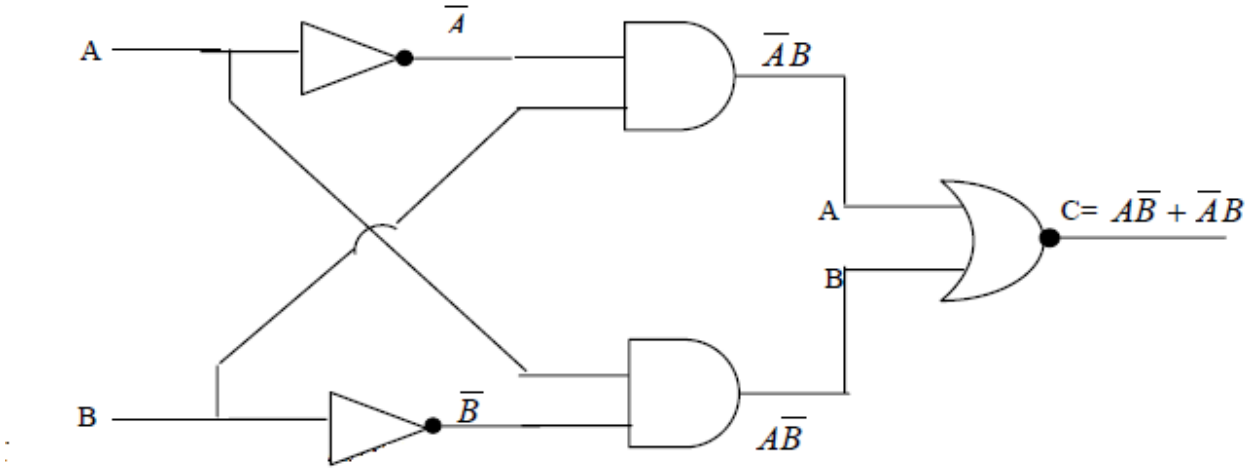
NOR GATE:



A	B	$\overline{A+B}$
0	0	0
0	1	1
1	0	1
1	1	1

3) Realization of Basic EXOR Gate Using AND &NOR Gate

Basic configuration of EX-OR gate:



CONCLUSION:-

The truth tables for various digital gates like AND, OR, NAND, NOT, EX-OR, NOR are verified USING Universal gates.

EXPERIMENT NO. 3

EXPERIMENT NO.3

NAME:-

Study of TTL Clock Generation Using NAND Gates

OBJECTIVE:-

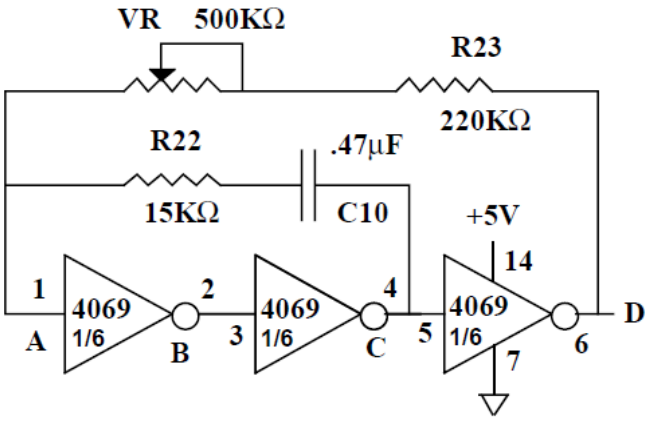
To design and verify TTL clock generation using NAND gates.

EQUIPMENTS:-

- BEL-DIT Board
- Power supply

THEORY

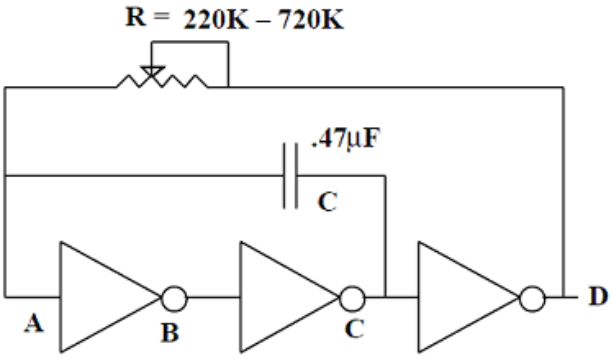
As we will see, many digital circuits require a “clock” signal. That is, a periodic square wave at a controlled frequency. Therefore, the final resistor, capacitor, and inverter circuit that we will consider is the free running oscillator shown in Fig. below.



Note, in this circuit R22 is only added to the circuit in order to limit the currents flowing into the input protection diodes of the inverters. If we further simplify the problem by assuming that the input protection diodes will limit the voltage at all gate inputs to the range {–0.7V – +5.7V}, we can eliminate this resistor (for analysis purposes only). In addition, we can lump the fixed resistor R23 in with the variable resistor RV. By eliminating both R22 and R23 we end up with the simplified circuit shown in Fig. 8.7.

Let us consider the operation of the circuit shown in Fig. 8.7. The easiest way to understand the operation of a RC and inverter circuit is to assume a given starting state and then simply analyze the operation of the circuit over time. For simplicity, let us assume that A=0V, B=5V, C=0V, and D=5V. Note, you can choose any set of starting voltages you wish. However, it is important to pick ones that are consistent with the inverter’s characteristic. Next consider how the circuit will evolve. As long as the voltage at point Are mains below 2.5V, then point C will stay at ground. In this case we have an R in series with a C to ground just as we did in below Fig. The solution for the voltage at point A follows from that derivation and is

$$V_A = 5V\ (1-e^{-t/RC})$$

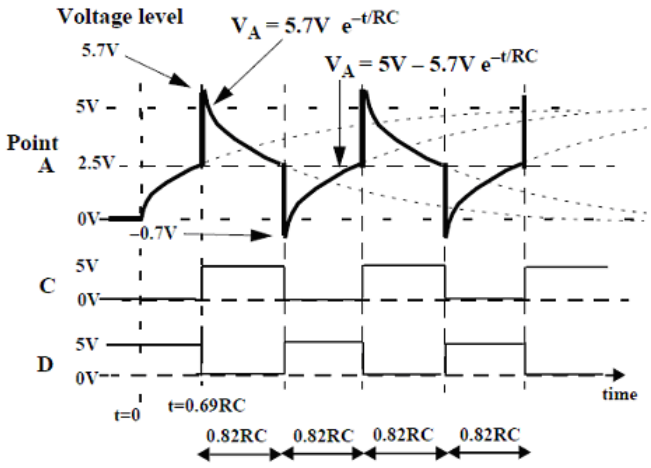


A simplified version of the clock circuit eliminating the nonessential resistors.

The first part of the curve for point A in above Fig. shows this curve. However, as soon as the voltage at point A crosses 2.5V the output at point D will go to 0V and at point C will go to +5V. Since the capacitor keeps the voltage across itself constant, the voltage at point A will try to go to 2.5V+5V=7.5V. However, the input protection diodes will clamp the voltage at point A at 5.7V (see above Fig.). Because point D is now 0V, the resistor will try to drag point A down to 0V from its starting point at 5.7V. Again, as soon as it crosses 2.5V the outputs of the inverters will all switch. An important question in determining the frequency of oscillation of this clock circuit is how long does it take to go from 5.7V to2.5V? We know the equation of this line is simply,

$$V_A(t) = 5.7\ V\ e^{-t/RC} = 2.5V$$

We can set it equal to 2.5V and solve for the value of t. In this case it takes 0.82RC units of time before the inverters switch. Note the product of resistance and capacitance has units of time. Specifically, 1 farad times 1 ohm is equal to 1 second.



Voltage waveforms over time for the RC + inverters clock generator.

As soon as the voltage at point A crosses 2.5V the output at point D will go to 5V and at point C will go to 0V. Since the capacitor keeps the voltage across itself constant, the voltage at point A will try to go to +2.5V–5V=–2.5V. However, the input protection diodes will clamp the voltage at point A at –0.7V (see above Fig.). Because point D is now at +5V, the resistor will try to drag point A up to +5V from its starting point at –0.7V. Again, as soon as it crosses 2.5V the outputs of the inverts will all switch. How long does it take to go from –0.7V to 2.5V? We know the equation of this line is simply

$$V_A(t) = 5.7\ V\ e^{-t/RC} = 2.5V$$

We can set it equal to 2.5V and solve for the value of t. In this case it again takes 0.82RC units of time before the inverters switch. Therefore the period of this oscillator is 1.64RC seconds and its frequency of oscillation is 1/ (1.64RC) Hz.

PROCEDURE:-

1) Do the Connection as shown in figure below. To mount R& C make use of ZIF socket
2) Observe the output at point D on oscilloscope.
3) Verify the theoretical and practical frequency of clock.
4) Observe the effect on frequency of clock by varying the resistor value.

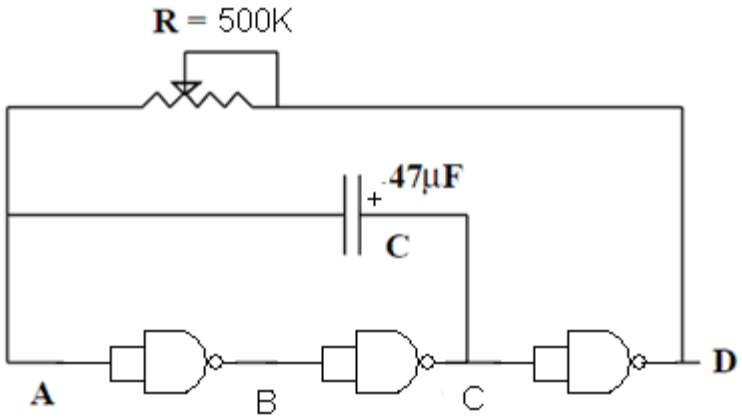
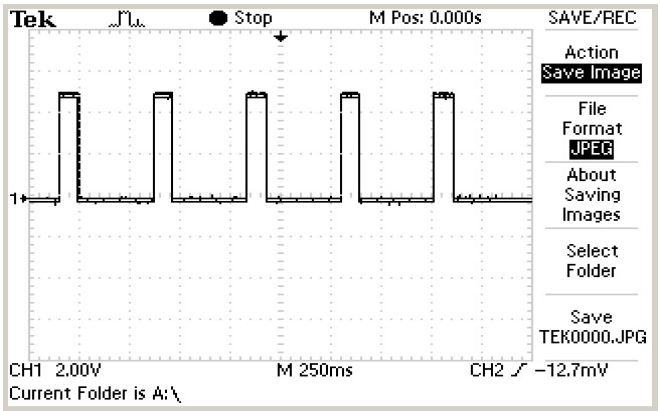


FIG: Digital Clock generation using gates

CONCLUSION:-

Effect on frequency of clock by varying the resistor value is observed.

WAVEFORM:-



EXPERIMENT NO. 4

EXPERIMENT NO.4

NAME:-

Study of Key DEBOUNCE Circuit

OBJECTIVE:-

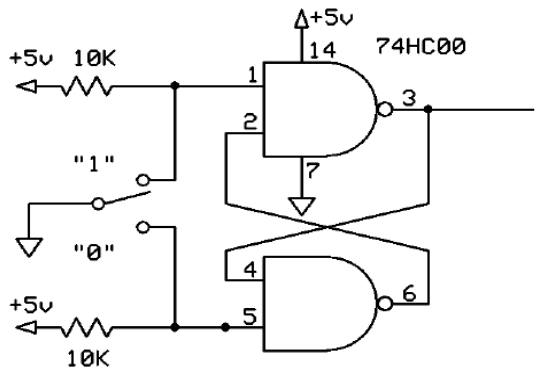
To study and verify key denounce using NAND Gates.

EQUIPMENTS:-

- BEL-DIT Board
- Power supply

THEORY:-

With any mechanical switch, the contacts rarely open and close cleanly. Rather, when you close a switch, the contacts are driven together and when they make contact, they bounce back apart, close again, bounce apart again, etc. Eventually, the bouncing ceases and the contacts remain closed. A similar phenomenon occurs when you open the contacts
The duration of the contact bounce depends on the type of switch, the quality of the materials and workmanship, the condition of the contacts and other factors but a good rule of thumb is that the duration will be in the range of 30mS to50mS. Depending on the timing, this often produces undesirable results. For example, say your program is keeps track of how many times a push button switch closes and increments a counter. The false closures due to contact bounce will likely lead to erroneous Counts.
So, what can be done about contact bounce? The methods employed to deal with this phenomenon are called denouncing and it can be done in hardware or in software. To do so in software is fairly simple: sample the switch state and when a change is detected, delay for 30mS to 50mS before proceeding. That simple delay helps to ensure that your program won't get back around to testing the switch state again until after the bouncing has ceased. The software denounce method works well for many situations but for some it does not.
For example, if the flow of the software cannot tolerate the required delay it may be necessary to implement the denouncing in hardware. The schematic below Figure shows a simple switch denouncing circuit.



This circuit employs a single pole, double throw (SPDT) switch and two NAND gates connected as an S-R flip-flop. The switch's two positions are labeled relative to the logic signal generated by the upper NAND gate. Of course, the lowerNAND gate's output will be the complement (opposite logic level) of that of the upper gate.

PROCEDURE:-

- Do the connection as shown in figure below. To mount R & switch make use of ZIF Socket.
- Press the switch and observe the output of NAND gate in DSO.
- Apply this Key denounced signal to counter and observe that the bounce effect of
- key is nullifying using gates.

EXPERIMENT NO. 5

EXPERIMENT NO.5

NAME:-

Study of De-Morgan's Theorem

OBJECTIVE:-

To study and verify DeMorgan's theorem

EQUIPMENTS:-

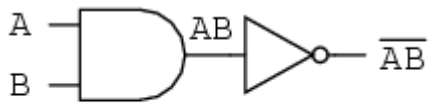
- BEL-DIT Board
- Power supply

THEORY:-

A mathematician named De Morgan developed a pair of important rules regarding group complementation in Boolean algebra. By group complementation, represented by a long bar over more than one variable.

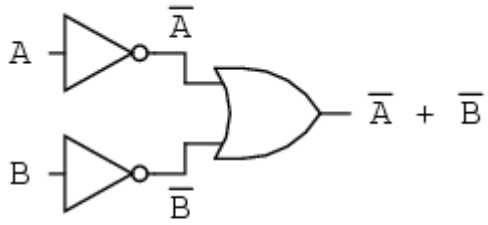
Inverting all inputs to a gate reverses that gate's essential function from AND to OR, or vice versa, and also inverts the output. So, an OR gate with all inputs inverted (a Negative-OR gate) behaves the same as a NAND gate, and an AND gate with all inputs inverted (a Negative-AND gate) behaves the same as a NOR gate. De Morgan's theorems state the same equivalence in "backward" form: that inverting the output of any gate results in the same function as the opposite type of gate (AND vs. OR) with inverted inputs:

A	**B**	$\overline{AB}$	$\overline{A}$	$\overline{B}$	$\overline{A}+\overline{B}$
0	0	1	1	1	1
0	1	1	1	0	1
1	0	1	0	1	1



1	1	0	0	0	0

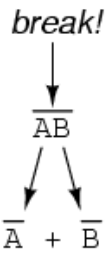
. . . is equivalent to . . .



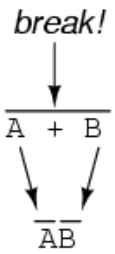
$$\overline{AB} = \overline{A} + \overline{B}$$

A long bar extending over the term AB acts as a grouping symbol, and as such is entirely different from the product of A and B independently inverted. In other words, (AB)' is not equal to A'B'. Because the "prime" symbol (') cannot be stretched over two variables like a bar can, we are forced to use parentheses to make it apply to the whole term AB in the previous sentence. A bar, however, acts as its own grouping symbol when stretched over more than one variable. This has profound impact on how Boolean expressions are evaluated and reduced, as we shall see.
DeMorgan's theorem may be thought of in terms of *breaking* a long bar symbol. When a long bar is broken, the operation directly underneath the break changes from addition to multiplication, or vice versa, and the broken bar pieces remain over the individual variables. To illustrate:

DeMorgan's Theorems

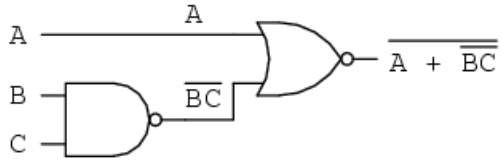


NAND to *Negative-OR*

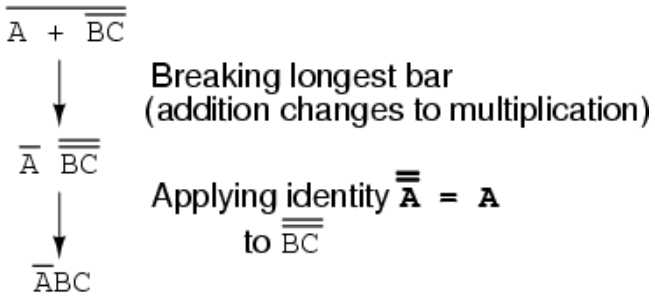


NOR to *Negative-AND*

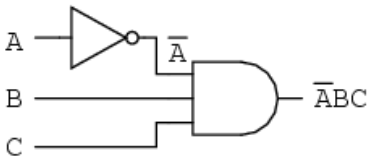
When multiple "layers" of bars exist in an expression, you may only break *one bar at a time*, and it is generally easier to begin simplification by breaking the longest (uppermost) bar first. To illustrate, let's take the expression (A + (BC)')' and reduce it using DeMorgan's Theorems:



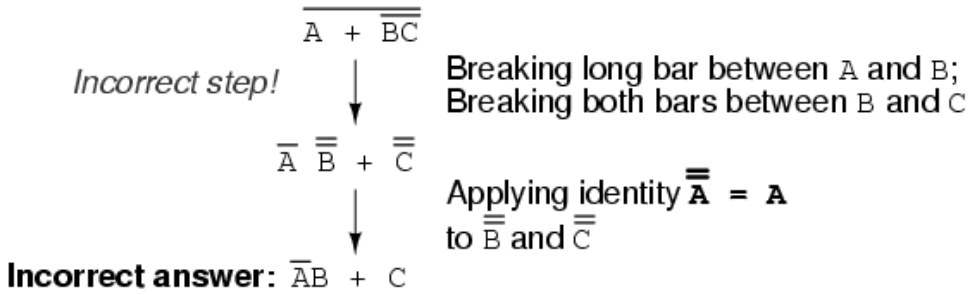
Following the advice of breaking the longest (uppermost) bar first, We'll begin by breaking the bar covering the entire expression as a first step:



As a result, the original circuit is reduced to a three-input AND gate with the A input inverted:

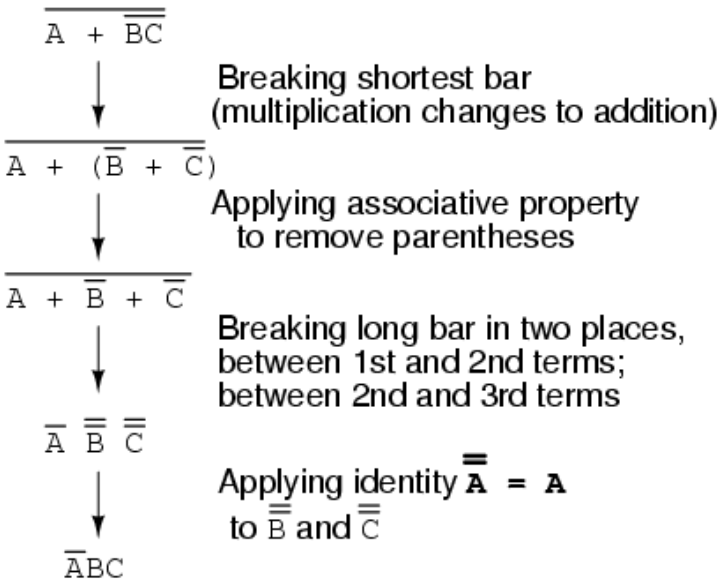


You should never break more than one bar in a single step, as illustrated here:



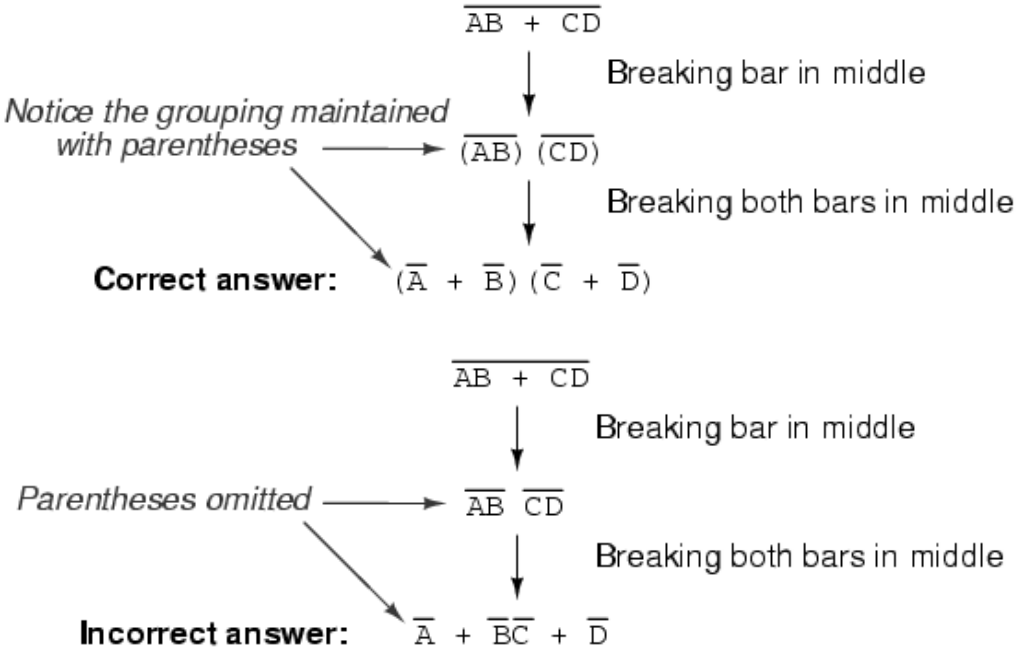
As tempting as it may be to conserve steps and break more than one bar at a time, it often leads to an incorrect result.

It is possible to properly reduce this expression by breaking the short bar first, rather than the long bar first



The end result is the same, but more steps are required compared to using the first method, where the longest bar was broken first

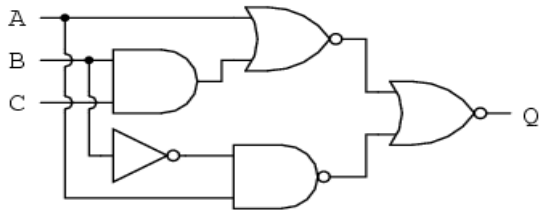
Consider this example, starting with a different expression:



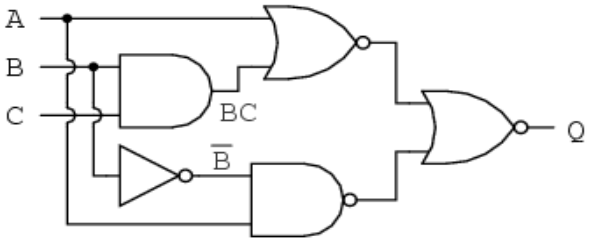
As you can see, maintaining the grouping implied by the complementation bars for this expression is crucial to obtaining the correct answer.

Verification Of De-Morgan's Theorem

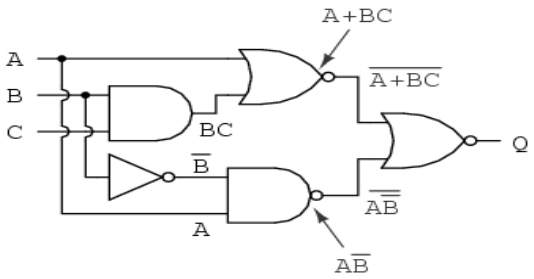
Let's apply the principles of DeMorgan's theorems to the simplification of a gate circuit:



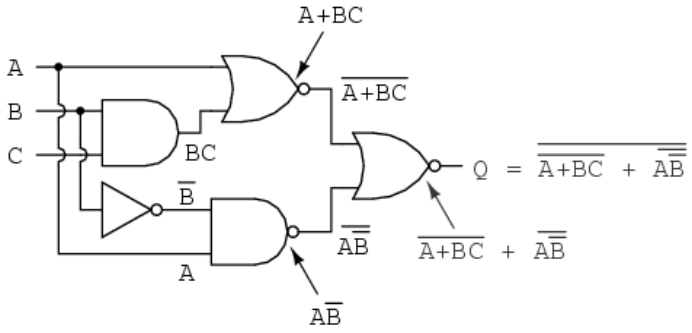
As always, our first step in simplifying this circuit must be to generate an equivalent Boolean expression. We can do this by placing a sub-expression label at the output of each gate, as the inputs become known. Here's the first step in this process:



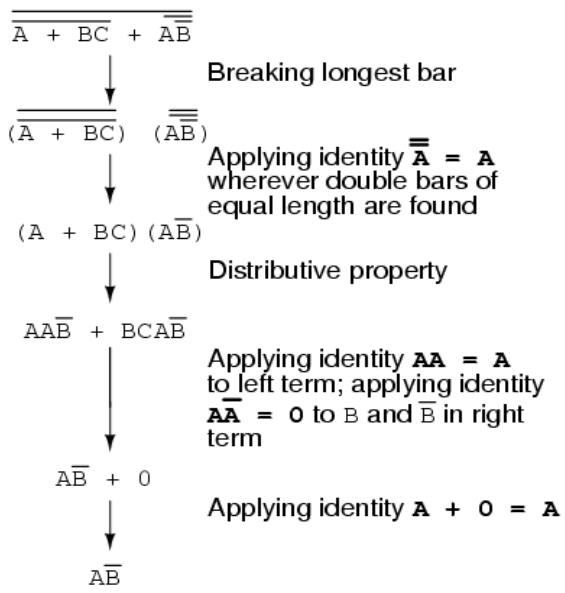
Next, we can label the outputs of the first NOR gate and the NAND gate. When dealing with inverted-output gates, we find it easier to write an expression for the gate's output without the final inversion, with an arrow pointing to just before the inversion bubble. Then, at the wire leading out of the gate (after the bubble), we write the full, complemented expression. This helps ensure we don't forget a complementing bar in the sub-expression, by forcing me to split the expression-writing task into two steps:



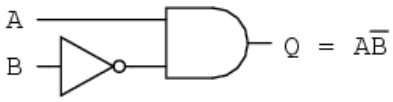
Finally, we write an expression (or pair of expressions) for the last NOR gate:



Now, we reduce this expression using the identities, properties, rules, and theorems (DeMorgan's) of Boolean algebra:



The equivalent gate circuit for this much-simplified expression is as follows:

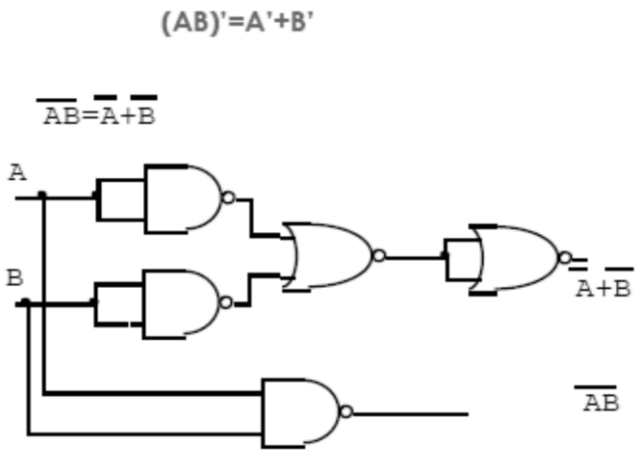


PROCEDURE:-

Verification of DeMorgan's theorem:

A) Theorem 1: (AB)' = A' + B'

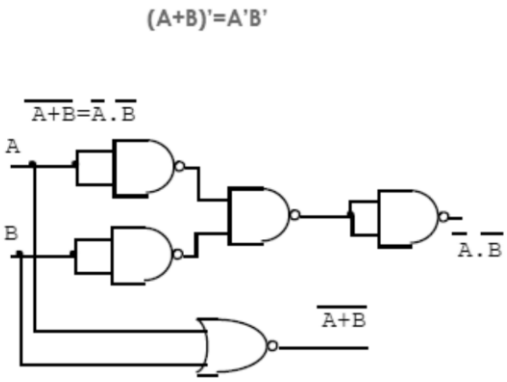
- Do the connection as shown in the circuit above.
- Connect A & B terminals to the Logic Inputs from I/P Switches.
- Connect both the outputs to led indicators in O/P section
- Provide Different combinations of inputs to A & B and observe the output on LEDs.
- For any combination of input, output at both the LED is similar.
- From this we can verify the DeMorgan's theorem. Calculate the gate count and chip count before and after applying DeMorgan's theorem.



Theorem 1) De-Morgan's theorem verification

B) Theorem 2: (A+B)' = A'B'

- Repeat the above procedure for below rule too



EXPERIMENT NO. 6

EXPERIMENT NO.6

NAME:-

Study of Boolean Expression Simplification

OBJECTIVE:-

To study the Boolean rules &Boolean expression simplification

EQUIPMENT:-

- BEL-DIT Board
- Power supply

THEORY:-

Here is the list of rules used for the Boolean expression simplifications.

The Idempotent Laws	AA = A	A+A = A
The Associative Laws	(AB) C = A (BC)	(A+B) +C = A+ (B+C)
The Commutative Laws	AB = BA	A+B = B+A
The Distributive Laws	A (B+C) = AB+AC	A+BC = (A+B) (A+C)
The Identity Laws	AF = F AT = A	A+F = A A+T = T
The Complement Laws	AA = F A+A = T	F = T T = F
The Involution Law	$\overline{\overline{A}}$ = A	
De-Morgan's Law	AB = A+BA+B = AB	

(T – True – 1 and F – False – 0)

Find below some examples of Boolean algebra simplifications. Each line gives a form of the expression, and the rule or rules used to derive it from the previous one. Generally, there are several ways to reach the result.

- Simplify: C + BC:

Expression	**Rule(s) Used**
C + BC	**Original Expression**

C + (B + C)	**DeMorgan's Law.**
(C + C) + B	**Commutative, Associative Laws.**
T + B	**Complement Law.**
T	**Identity Law.**

- Simplify: AB(A + B)(B + B):

Expression	**Rule(s) Used**
AB(A + B)(B + B)	Original Expression
AB(A + B)	Complement law, Identity law.
(A + B)(A + B)	DeMorgan's Law
A + BB	Distributive law. This step uses the fact that or distributes over and. It can look a bit strange since addition does not distribute over multiplication.
A	Complement, Identity.

- Simplify: (A + C)(AD + AD) + AC + C:

Expression	**Rule(s) Used**
(A + C)(AD + AD) + AC + C	Original Expression
(A + C)A(D + D) + AC + C	Distributive.
(A + C)A + AC + C	Complement, Identity.
A((A + C) + C) + C	Commutative, Distributive.
A(A + C) + C	Associative, Idempotent.
AA + AC + C	Distributive.

A + (A + T)C	Idempotent, Identity, Distributive.
A + C	Identity, twice.

- You can also use distribution of or over and starting from A(A+C)+C to reach the same result by another route.
- Simplify: A(A + B) + (B + AA)(A + B):

Expression	**Rule(s) Used**
A(A + B) + (B + AA)(A + B)	Original Expression
AA + AB + (B + A)A + (B + A)B	Idempotent (AA to A), then Distributive, used twice.
AB + (B + A)A + (B + A)B	Complement, then Identity. (Strictly speaking, we also used the Commutative Law for each of these applications.)
AB + BA + AA + BB + AB	Distributive, two places.
AB + BA + A + AB	Idempotent (for the A's), then Complement and Identity to remove BB.
AB + AB + AT + AB	Commutative, Identity; setting up for the next step.
AB + A(B + T + B)	Distributive.
AB + A	Identity, twice (depending how you count it).
A + AB	Commutative.
(A + A)(A + B)	Distributive.
A + B	Complement, Identity.

EXPERIMENT NO. 7

EXPERIMENT NO.7

NAME:-

Study of Adder and Subtractor

OBJECTIVE:-

To study the half adder, full adder, half Subtractor and full Subtractor

EQUIPMENTS:-

- BEL-DIT Board
- Power supply

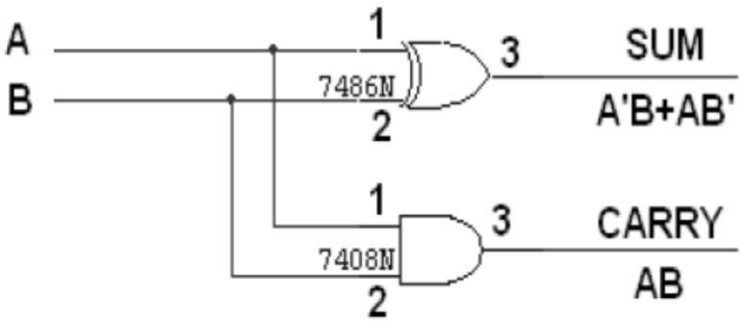
THEORY:-

Half Adder

A half adder has two inputs for the two bits to be added and two Outputs one from the sum ‘ S’ and other from the carry ‘ c’ into the higher adder position. Above circuit is called as a carry signal from the BEL-DIT on of the less significant bits sum from the X-OR Gate the carry out from the AND gate.

Logic Diagram of Half Adder

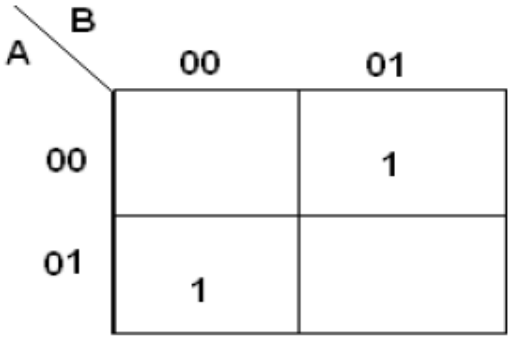
HALF ADDER



TRUTH TABLE:

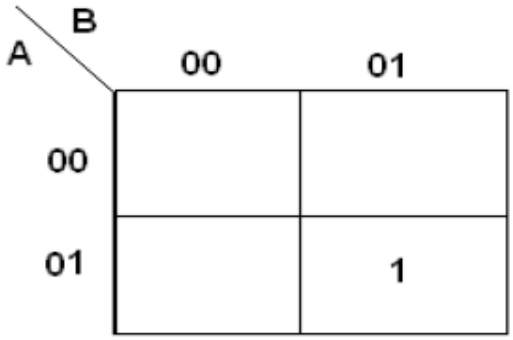
A	B	CARRY	SUM
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

K-Map for SUM:



SUM = A'B + AB'

K-Map for CARRY:



CARRY = AB

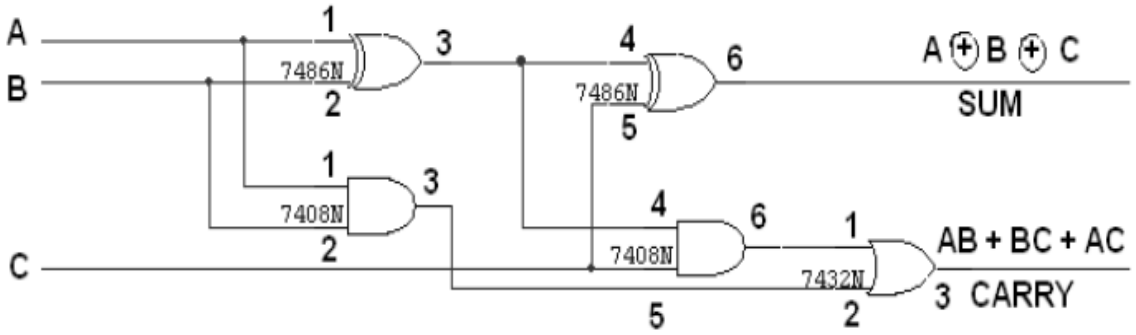
Full Adder

A full adder is a combinational circuit that forms the arithmetic sum of input; it consists of three inputs and two outputs. A full adder is useful to add three bits at a time but a half adder cannot do so. In full adder sum output will be taken from X-OR Gate, carry output will be taken from OR Gate.

Logic Diagram of Full adder

FULL ADDER

FULL ADDER USING TWO HALF ADDER



TRUTH TABLE:

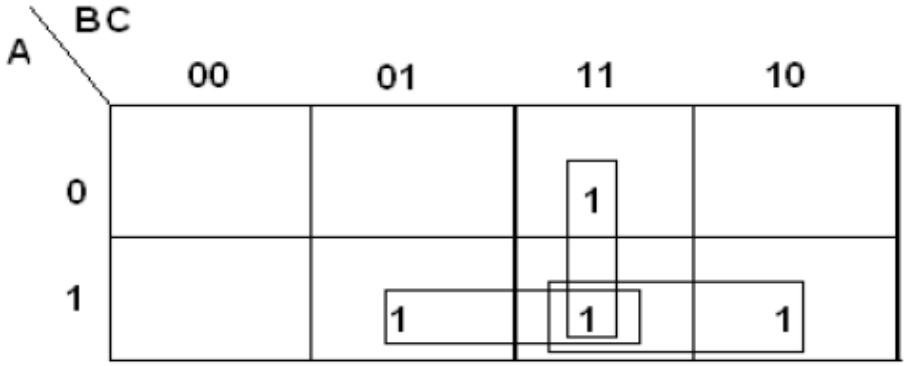
A	B	C	CARRY	SUM
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

K-Map for SUM:

A \ BC	00	01	11	10
0		1		1
1	1		1	

SUM = A'B'C + A'BC' + ABC' + ABC

K-Map for CARRY:

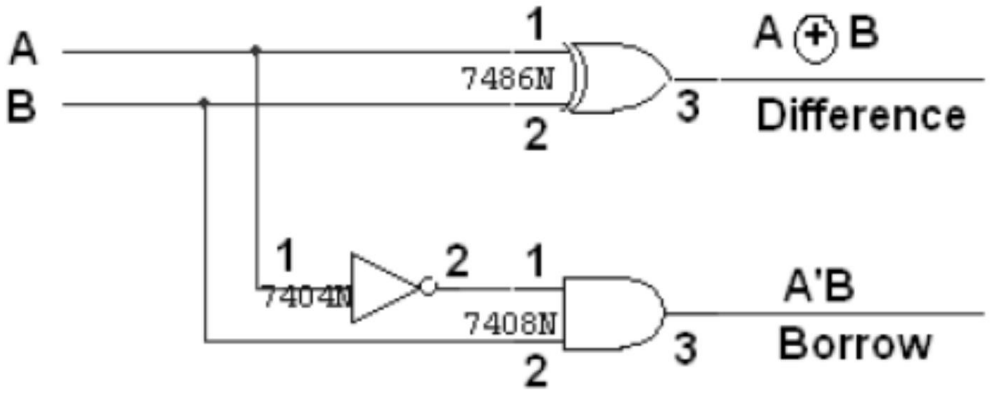


CARRY = AB + BC + AC

Half Subtractor

The half subtractor is constructed using X-OR and AND Gate. The half subtractor has two input and two outputs. The outputs are difference and borrow. The difference can be applied using X-OR Gate, borrow output can be implemented using an AND Gate and an inverter.

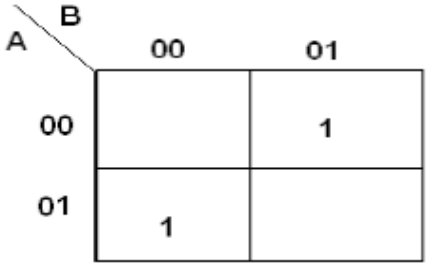
Logic Diagram of half Subtractor



TRUTH TABLE:

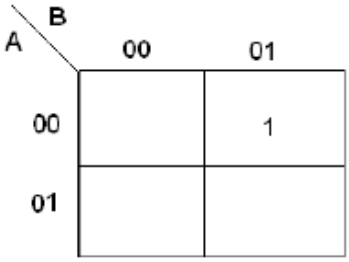
A	B	BORROW	DIFFERENCE
0	0	0	0
0	1	1	1
1	0	0	1
1	1	0	0

K-Map for DIFFERENCE:



DIFFERENCE = A'B + AB'

K-Map for BORROW:



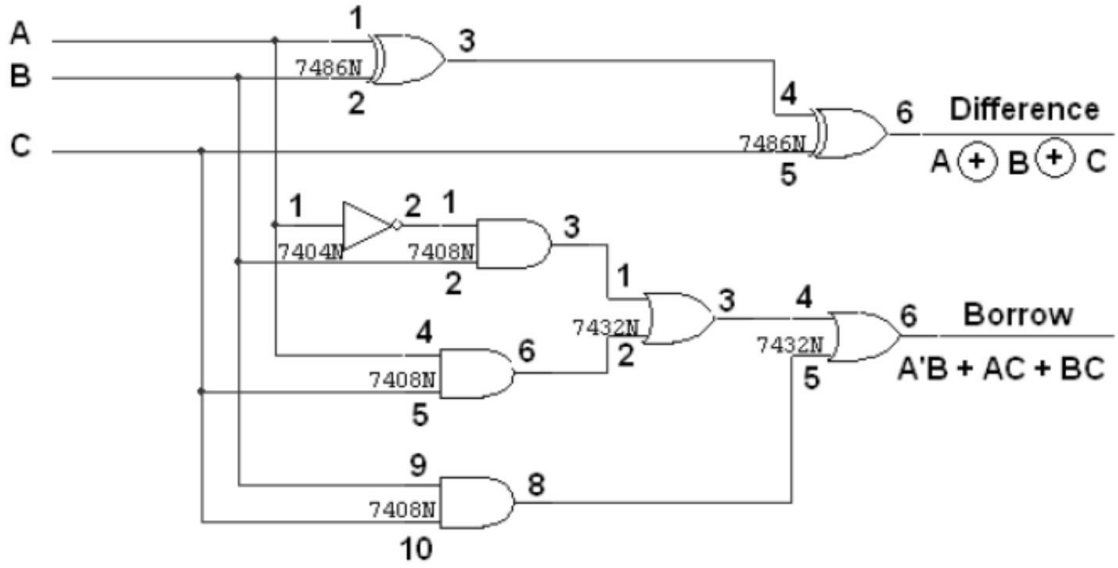
BORROW = A'B

Full Subtractor

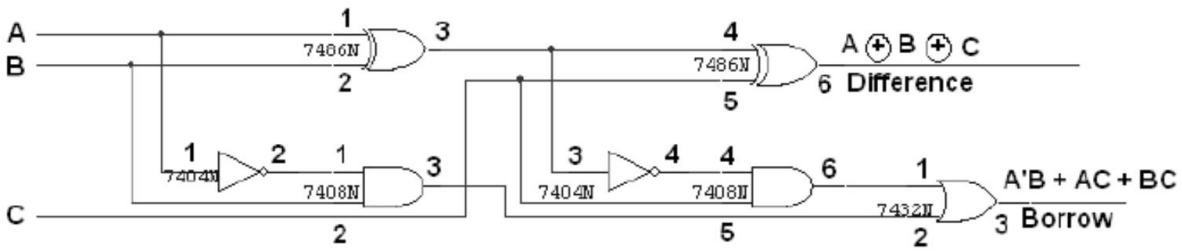
The full subtractor is a combination of X-OR, AND, OR, NOT Gates. In a full subtractor the logic circuit should have three inputs and two outputs. The two half subtractor put together gives a full subtractor .The first half subtractor will be C and A B. The output will be difference output of full subtractor. The expression AB assembles the borrow output of the half subtractor and the second term is the inverted difference output of first XOR.

Logic Diagram of Full Subtractor

FULL SUBTRACTOR



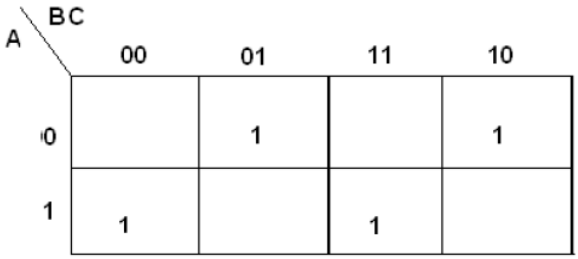
Full Subtractor using two half subtractor



TRUTH TABLE:

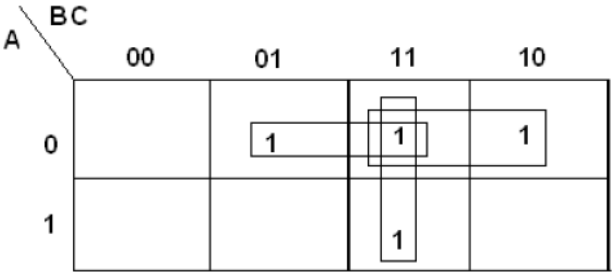
A	B	C	BORROW	DIFFERENCE
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	1	0
1	0	0	0	1
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

K-Map for Difference:



Difference = A'B'C + A'BC' + AB'C' + ABC

K-Map for Borrow:



Borrow = A'B + BC + A'C

PROCEDURE:-

B) Half Adder
- Connect A & B I/P of Half Adder to switches from input switches section.
- Connect SUM O/P of Half Adder to L1 & CARRY O/P of Half Adder to L2 in O/P SECTION.
- Switch on the power supply of the Kit.
- Provide proper inputs to half adder using switches as per truth table of half adder shown above.
- Observe the O/P of Half Adder on LEDs.
- Verify the functionality of half Adder as per truth table & Note it down.

C) Full Adder
- Connect A, B and C I/P of Full adder to switches from input switches section.
- Connect SUM & CARRY O/P of Full Adder to LEDs from O/P LED section.
- Switch ON the power supply of the Kit.
- Provide proper inputs to Full adder using switches as per truth table ofFull adder shown above.
- Observe the O/P of Full Adder on LEDs.
  Verify the functionality of Full Adder as per truth table & Note it down.

D) Half Subtractor
- Connect A & B I/P of Half Subtractor to switches from input switches section.
- Connect DIF&BOR O/P of Half subtractor to LEDs in O/P LED section.
- Switch on the power supply of the Kit.
- Provide proper inputs to half subtractor using switches as per truth table of half adder shown above.
- Observe the O/P of Half subtractor on LEDs.
- Verify the functionality of half subtractor as per truth table & Note it down.

E) Full Subtractor
- Connect A, B & C I/P of Full Subtractor to switches from input switches section.
- Connect DIF&BOR O/P of Full Subtractor to LEDs from O/P SECTION.
- Switch ON the power supply of the Kit.
- Provide proper inputs to Full Subtractor using switches as per truth table of Full Subtractor shown above.
- Observe the O/P of Full Subtractor on LEDs.
- Verify the functionality of Full Subtractor as per truth table & Note it down.

EXPERIMENT NO. 8

EXPERIMENT NO.8

NAME:-

Study of Flip-flops

OBJECTIVE:-

To study the behavior of S-R, J-K, MS-JK, D and T Flip Flop

EQUIPMENTS:-

- BEL-DIT Board
- Power supply

THEORY:-

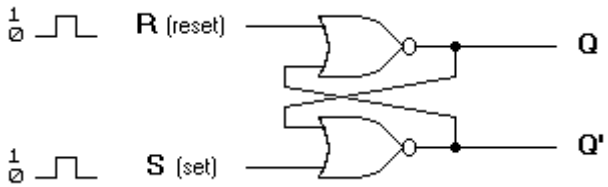
S-R Flip-Flop

A flip-flop circuit can be constructed from two NAND gates or two NOR gates. These flip-flops are shown in Figure 1and Figure 2. Each flip-flop has two outputs, Q and Q', and two inputs, set and reset. This type of flip-flop is referred to as an SR flip-flop or SR latch.

The flip-flop in Figure 1 has two useful states. When Q=1 and Q'=0, it is in the set state (or 1-state).

When Q=0 and Q'=1, it is in the clear state (or 0-state). The outputs Q and Q' are complements of each other and are referred to as the normal and complement outputs, respectively. The binary state of the flip-flop is taken to be the value of the normal output.

When 1 is given as input to both the set and reset inputs of the flip-flop in Figure 1, both Q and Q' outputs go to 0. This BEL-DIT violates the fact that both outputs are complements of each other. In normal operation this BEL-DIT must be avoided by making sure that 1's are not applied to both inputs simultaneously.

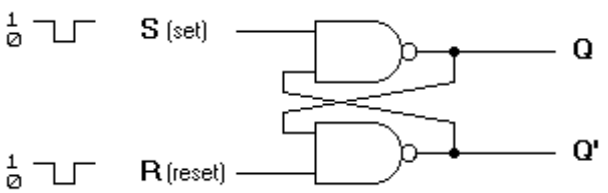


(a) Logic diagram

S R	Q Q'	
1 0	1 0	
0 0	1 0	(after S=1, R=0)
0 1	0 1	
0 0	0 1	(after S=0, R=1)
1 1	0 0	

(b) Truth table

Figure 1. Basic flip-flop circuit with NOR gates



(a) Logic diagram

S R	Q Q'	
1 0	0 1	
1 1	0 1	(after S=1, R=0)
0 1	1 0	
1 1	1 0	(after S=0, R=1)
0 0	1 1	

(b) Truth table

Figure 2. Basic flip-flop circuit with NAND gate

The NAND basic flip-flop circuit in Figure 2(a) operates with inputs normally at 1 unless the state of the flip-flop has to be changed. A 0 is applied momentarily to the set input causes Q to go to 1 and Q' to go to 0, putting the flip-flop in the set state. When both inputs go to 0, both outputs go to 1. This condition should be avoided in normal operation.

JK Flipflop

Fig 3(a) shows the clocked J-K flip-flop with clear (CR) and preset (PR) inputs. The small circle (inversion symbols) on these inputs indicates that logic '0' is required to clear or set the flip-flop. Thus the '0' applied to the clear input will reset the flip-flop to Q = '0', and a '0' applied to the Preset input will set the flip-flop to Q = '1'. These inputs override the clock & J-K input. I.e. a '0' applied to the clear input will reset the flip-flop regardless of the values of J-K, and the clock. Under normal condition, a '0' should not be applied simultaneously to clear and preset. When the clear and preset inputs are both held at logic '1', the J, K and clock inputs operate in the normal manner. Fig.3 (b) shows the truth table for J-K Flip-Flop.

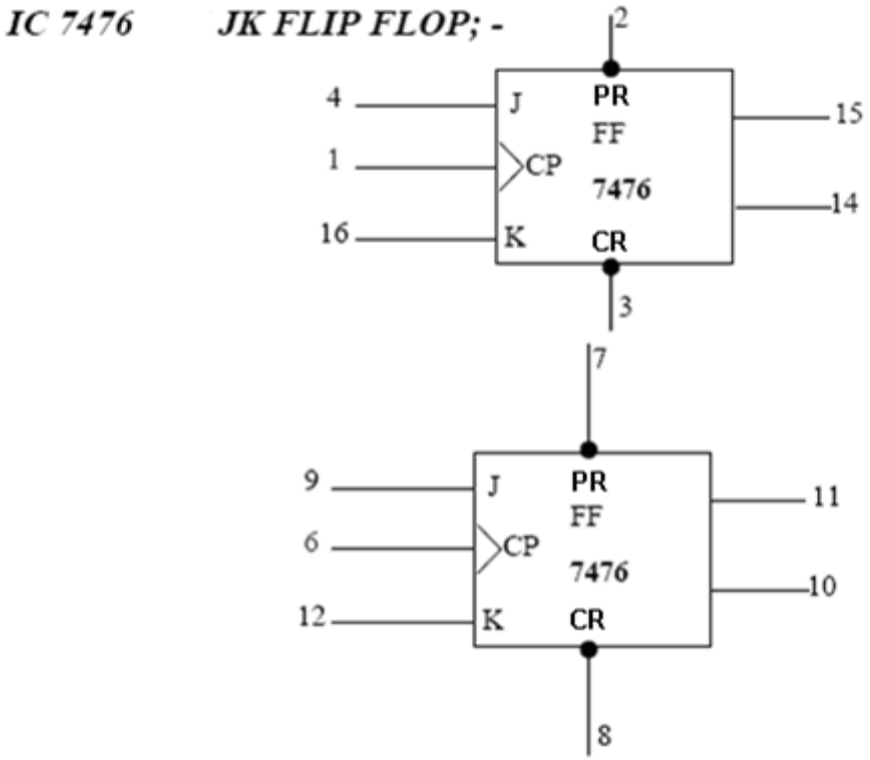


Fig.3 (a)

TRUTH TABLE FOR JK-FLIP FLOP (IC 7476); -

SD Preset	CD Clear	Clock	J	K	OUTPUTS	
					Q	$\bar{Q}$
L	H	X	X	X	H	L
H	L	X	X	X	L	H
L	L	X	X	X	H*	H*
H	H	↓	L	L	Q_0	$\bar{Q}_0$
H	H	↓	H	L	H	L
H	H	↓	L	H	L	H
H	H	↓	H	H	TOGGLE	
H	H	H	X	X	Q_0	$\bar{Q}_0$

*Unstable condition. It will not remain after C_n and P_n inputs return to their inactive (high) state

Fig.3 (b)

Master –Slave JK Flip-Flop

J-K Flip-Flop suffers from timing problems called "race" if the output Q changes state before the timing pulse of the clock input has time to go "OFF". To avoid this the timing pulse period (T) must be kept as short as possible (high frequency). As this is sometimes is not possible with modern TTL IC's the much improved **Master-Slave JK Flip-flop** was developed. This eliminates all the timing problems by using two SR flip-flops connected together in series, one for the "Master" circuit, which triggers on the leading edge of the clock pulse and the other, the "Slave" circuit, which triggers on the falling edge of the clock pulse.

The **Master-Slave Flip-Flop** is basically two JK bistable flip-flops connected together in a series configuration with the outputs from Q and Q' from the "Slave" flip-flop being fed back to the inputs of the "Master" with the outputs of the "Master" flip-flop being connected to the two inputs of the "Slave" flip-flop as shown below.

Master-Slave JK Flip-Flops

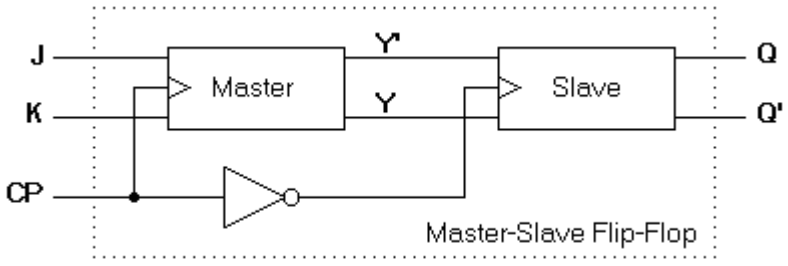


FIG. 3(c) logic diagram for master slave JK flip-flop

Fig.3(c) shows one way to build a JK master-slave flip-flop. It provides another way to avoid racing. Here's how it works. To begin with, the master is positive edge triggered and the slave, negative edge triggered. Therefore, the master responds to its J and k inputs before the slave.

- If J ='1' and K = '0', the master set on positive clock edge. The high Q output of the master drives the J input of the slave, so when the negative clock edge hits, the slave sets, copying the action of master.
- If J = '0' and K = '1', the master reset on leading edge of the clock. The high 'Q' output of the master goes to the K input of the slave. Therefore, the arrival of the clock's trailing edge forces the slave to reset. Again, the slave has copied the master.
- If the master's J and K input are both High. It toggles on the positive edge and the slave then toggles on the negative clock edge. Regardless of what the master does, therefore, the slave copies it: if the master sets, the slave sets; if the master reset, the slave resets.

The timing relationship is shown in Fig.3 (d) and is assumed that the flip-flop is in the clear state prior to the occurrence of the clock pulse. The output state of the master-slave flip-flop occurs on the negative transition of the clock pulse. Some master-slave flip-flops change output state on the positive transition of the clock pulse by having an Additional inverter between the CP terminal and the input of the master.

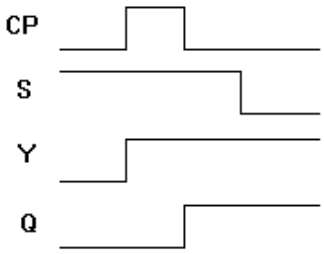
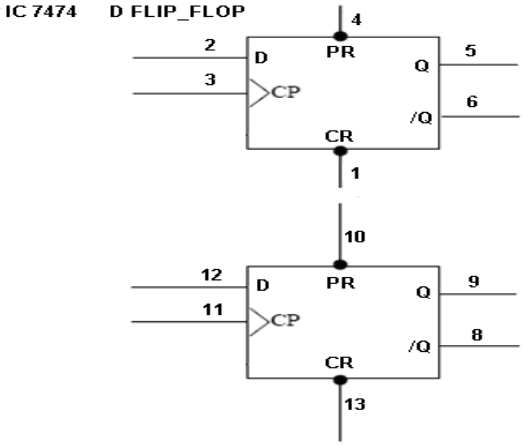


Fig.3 (d) Timing relationship in a master slave flip-flop

D Flip-Flop

As seen from the truth table Fig.8 (f), the state of this flip-flop after the clock pulse $Q^{(t+1)}$ is equal to the input D before the clock pulse. For example, if D = '1' before the clock pulse, Q = '1' after the clock pulse regardless of the previous value of Q. therefore, the characteristic equation is $Q^{(t+1)} = D$.



Q	D	Q(t+1)
0	0	0
0	1	1
1	0	0
1	1	1

Fig. 3(E) Block diagram of D flip-flop **Fig.3 (F) truth table for D flip-flop**

T Flip-Flop

T flip-flop is a clocked Flip-Flop whose output "toggles", i.e. changes to the complementary logic state, on every active transition of the clock signal. The device acts as a divide-by-two counter since two active transitions of the clock signal generate one active transition of the output. It can be considered as being equivalent to a J-K Flip-Flop who's J and K inputs are held at logic 1.

This type of flip-flop is a simplified version of the JK flip-flop. It is not usually found as an IC chip by itself, but is used in many kinds of circuits, especially counter and dividers. Its only function is that it toggles itself with every clock pulse (on either the leading edge, on the trailing edge) it can be constructed from the JK flip-flop as shown in Fig.3 (H).

This flip flop is normally set, or "loaded" with the preset and clear inputs. It can be used to obtain an output pulse train with a frequency of half that of the clock pulse train, as seen from the timing diagram, in this example, the T flip flop is triggered on the falling edge of the clock pulse.

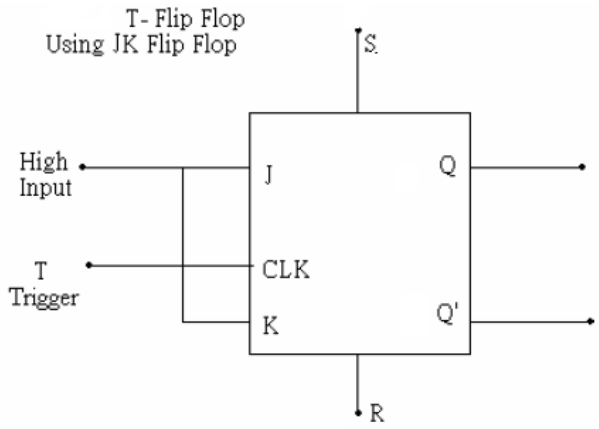
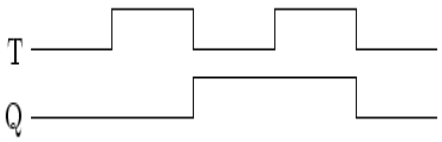


FIG. 3(G) T Flip-Flop using JK flip-Flop



Timing Diagram for T Flip-Flop.

J & K	S	R	CLOCK (T)	Q	Q'	COMMENT
1	0	0	X	1	1	Race
1	0	1	X	1	0	Set
1	1	0	X	0	1	Reset
1	1	1	↑	Q'_{n-1}	Q_{n-1}	Data Transfer

FIG.3 (H) truth Table for T Flip-Flop

PROCEDURE:-

A) S-R Flip-Flop

- Connect the logic signals from the logic input switches to S and R input of R-S Flip-Flop.
- Observe the logic outputs on the LEDs in O/P section.
- Verify the truth table of RS flip-flops.

B) J-K Flip-Flop

Note: use any one of the gate available out of 4.

- Connect PR to **PRESET**, CR to **CLEAR** and J and K terminals to the logic input switches.
- Connect CLK of JK flip-flop to Clock terminal.
- Connect Q and /Q terminals to LED indicators in O/P section.
- Set the PR, CR,CLK, J and K Signals by means of the switches as per the truth table of JK flip-flop given above and verify the Q and/Q outputs by changing possible input condition.

C) Master Slave J-K Flip-Flop

Note: use any two of the gate available out of 4.

- Do the connection for MS JK Flip-Flop as shown in Fig.3(c) above.
- Connect PR to **PRESET**, CR to **CLEAR** of both the flip-flops and J and K terminals of master flip-flop to the logic input switches.
- Connect CLK of master JK flip-flop to Clock terminal.
- Connect Q and /Q terminals of slave flip-flop to LED indicators in O/P LED section. Also connect Q & /Q terminals of master flip-flop to the LEDs in O/P LED section.
- Set the PR, CR, clk, J and K Signals by means of the switches as per the truth table of MS JK flip-flop given above and verify the Q and /Q outputs.

D) D Flip-Flop

Note: use any one of the gate available out of 2.

- Connect PR to **PRESET**,CR to **CLEAR** and D terminals to the logic input switch.
- Connect the CLK of D Flip-Flop to CLOCK terminal.
- Connect Q and /Q terminals to LED indicators in O/P LED section.
- Set the PR, CR, CLK and D Signals by means of the switches as per the truth table of D flip-flop given above and verify the Q and /Q outputs.

E) T Flip-Flop

Note: use any one of the gate available out of 4.

- Do the connection for T Flip-Flop as shown in FIG.3 (g)above. Connect PR to **PRESET**, CR to **CLEAR** and T terminals to the logic input switch.
- Connect the CLK of T Flip-Flop to CLOCK terminal.
- Connect Q and /Q terminals to LED indicators in O/P LED section.
- Set the PR, CR, CLK and T Signals by means of the switches as per the truth table of T flip-flop given above and verify the Q and /Q outputs.

EXPERIMENT NO. 9

EXPERIMENT NO.9

NAME:-

Study of Counters

OBJECTIVE:-

To study and design of ripple (Asynchronous) counter using JK flip-flop

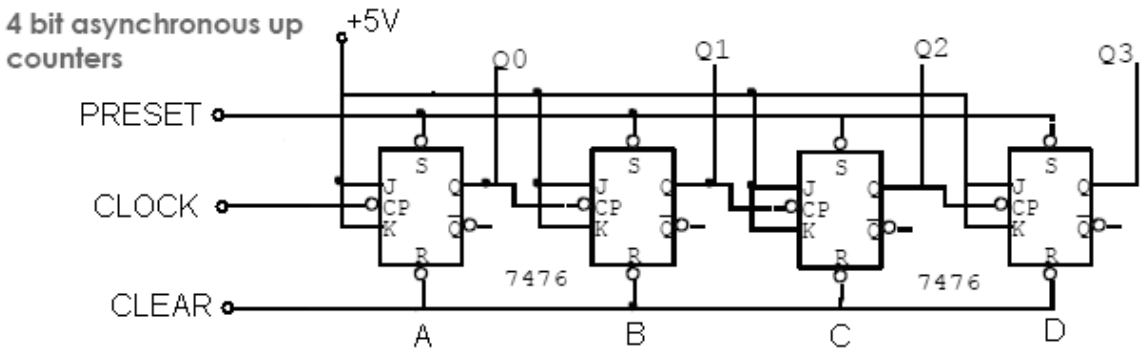
To study and design of Ring counter & Johnson counter.

EQUIPMENTS:-

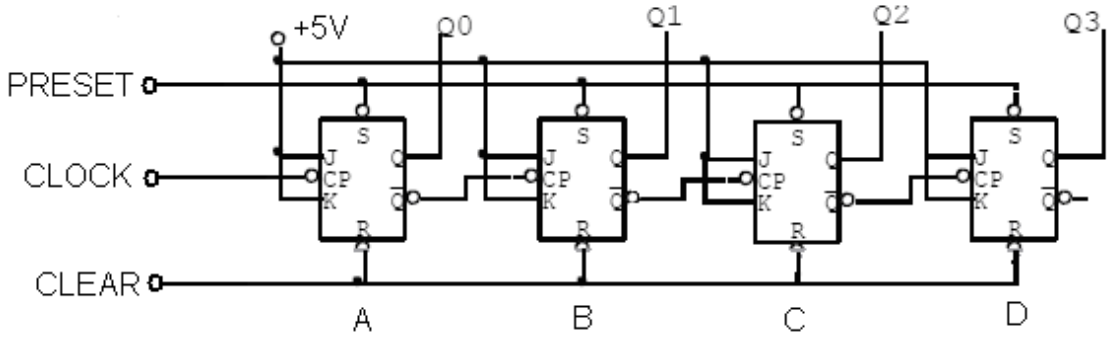
- BEL-DIT Board
- Power supply

THEORY:-

Asynchronous Counter: When the output of a flip-flop is used as a clock input for the next flip-flop, we call the counter a ripple counter or asynchronous counter. The A flip-flop must change states before it can trigger B flip-flop, and the B flip-flop has to change states before it can trigger the C Flip-flop and so on. The triggers move through the flip-flop like a ripple in water. Because of this, the overall propagation delay time is the sum of the individual delays. For instance, if each flip-flop in this four flip-flop counter has a propagation delay time of 10ns, the overall propagation delay time for the counter is 40ns.



4 bit asynchronous down counters

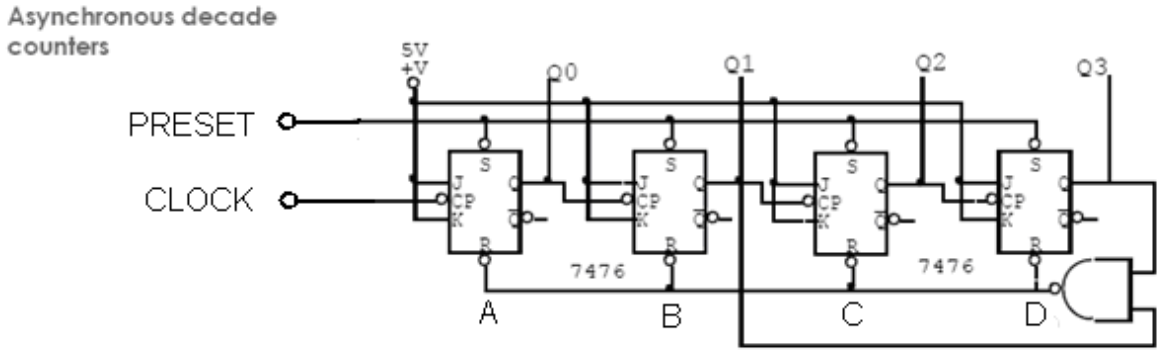


Function table for Asynchronous UP / DOWN Counter

CLK	UP Counter					Down Counter			
	Q3	Q2	Q1	Q0		Q3	Q2	Q1	Q0
0	0	0	0	0		1	1	1	1
1	0	0	0	1		1	1	1	0
2	0	0	1	0		1	1	0	1
3	0	0	1	1		1	1	0	0
4	0	1	0	0		1	0	1	1
5	0	1	0	1		1	0	1	0
6	0	1	1	0		1	0	0	1
7	0	1	1	1		1	0	0	0
8	1	0	0	0		0	1	1	1
9	1	0	0	1		0	1	1	0
10	1	0	1	0		0	1	0	1
11	1	0	1	1		0	1	0	0
12	1	1	0	0		0	0	1	1
13	1	1	0	1		0	0	1	0
14	1	1	1	0		0	0	0	1
15	1	1	1	1		0	0	0	0

Decade Counter: A Four Flip-flop has a natural count of 16. We can thus construct any counter that has a modulus between 16 and 2, inclusive. We might choose to use four flip-flops only for counters having a modulus between 16 and 9, since only three flip-flops are required for a modulus of less than 8, and only two are required for a modulus of less than 4.

There are a many different methods for constructing a counter having a modified count. A counter can be synchronous, asynchronous, or a combination of these two types; furthermore, there is the decision of which count is skip.



Referring the function table of the decade counter it mainly counts 10 states from 0000 to 1001. As soon as counter reaches to 1010 all flip-flops are get reset due to AND gate and counters start counting again from 0000.

Function table for Asynchronous UP / DOWN Counter:

CLK	UP Counter			
	Q3	Q2	Q1	Q0
0	**0**	**0**	**0**	**0**
1	**0**	**0**	**0**	**1**
2	**0**	**0**	**1**	**0**
3	**0**	**0**	**1**	**1**
4	**0**	**1**	**0**	**0**
5	**0**	**1**	**0**	**1**
6	**0**	**1**	**1**	**0**
7	**0**	**1**	**1**	**1**
8	**1**	**0**	**0**	**0**
9	**1**	**0**	**0**	**1**
10	0	0	0	0
11	0	0	0	1
12	0	0	1	0
13	0	0	1	1
14	0	1	0	0
15	0	1	0	1

- To design 4 bit Ring counter using JK flip-flop.

A **Ring Counter** is a type of counter composed of a circular Shift register. The output of the last shift register is fed to the input of the first register.
There are two types of ring counters:

- A straight ring counter or Over beck counter connects the output of the last shift register to the first shift register input and circulates a single one (or zero) bit around the ring. For example, in a 4-register counter, with initial register values of 1000, the repeating pattern is: 1000, 0100, 0010, 0001, 1000... . **Note that one of the registers must be pre-loaded with a 1 (or 0) in order to operate properly.**
- A twisted ring counter or Johnson counter connects the complement of the output of the last shift register to its input and circulates a stream of ones followed by zeros around the ring. For example, in a 4-register counter, with initial register values of

0000, the repeating pattern is: 0000, 1000, 1100, 1110, 1111, 0111, 0011, 0001, 0000... .

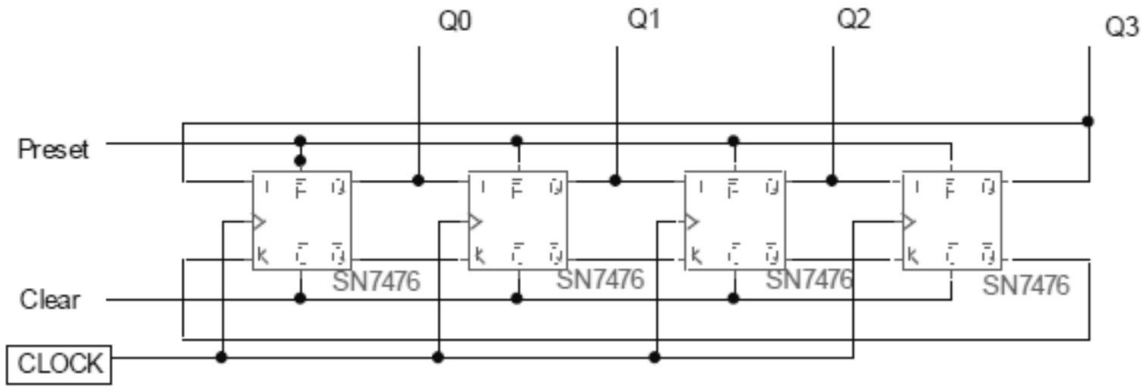
Ring Counter Sequence:

STATES	Q 0	Q1	Q2	Q3
0	1	0	0	0
1	0	1	0	0
2	0	0	1	0
3	0	0	0	1
0	1	0	0	0
1	0	1	0	0
2	0	0	1	0
3	0	0	0	1
0	1	0	0	0

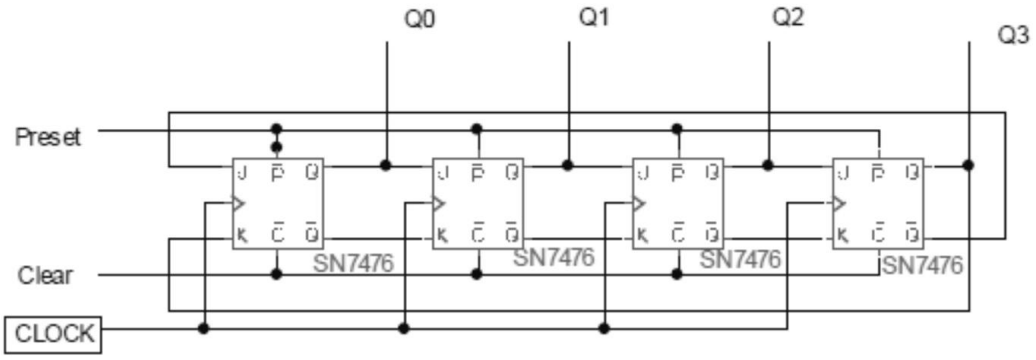
Twisted Ring / Johnson ring Counter Sequence:

STATES	Q0	Q1	Q2	Q3
0	0	0	0	0
1	1	0	0	0
2	1	1	0	0
3	1	1	1	0
4	1	1	1	1
5	0	1	1	1
6	0	0	1	1
7	0	0	0	1
0	0	0	0	0

Ring counter using JKFF



Jhonson counter using JKFF



PROCEDURE:-

Assemble the different counters as shown in above diagrams using JK flip-flop and verify its functionality by referring its function table.

EXPERIMENT NO. 10

EXPERIMENT NO.10

NAME:-

Study of Binary Counter

OBJECTIVE:-

To study of synchronous binary counter using IC 74191.

EQUIPMENTS

- BEL-DIT Board
- Power supply

THEORY:-

The 74LS191 circuit is a synchronous, reversible, up/down counter. Synchronous operation is provided by having all flip-flops clocked simultaneously, so that the outputs change simultaneously when so instructed by the steering logic. This mode of operation eliminates the output counting spikes normally associated with asynchronous (ripple Clock) counters.

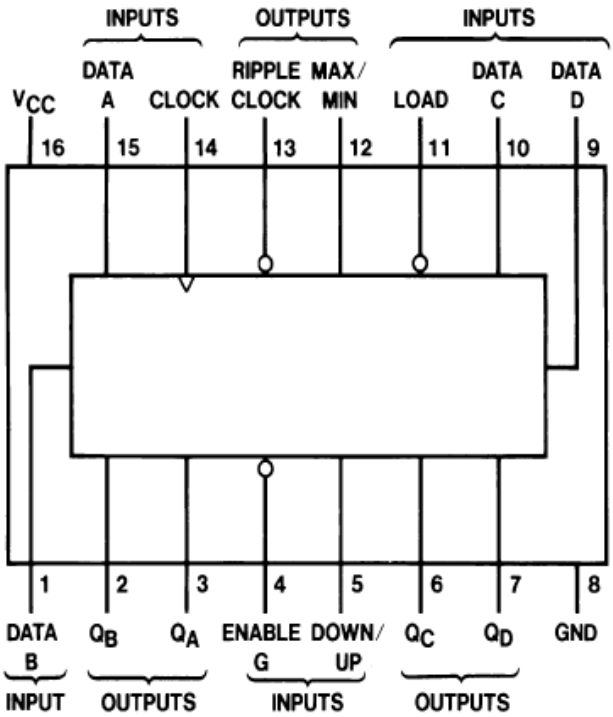
The outputs of the four internal master-slave flip-flops are triggered on a LOW-to-HIGH level transition of the clock input, if the enable input is LOW. A HIGH at the enable input inhibits counting. Level changes at either the enable input or the down/up input should be made only when the clock input is HIGH. The direction of the count is determined by the level of the down/up input. When LOW, the counter counts up and when HIGH, it counts down.

The counter is fully programmable; that is, the outputs maybe preset to either level by placing a LOW on the load input and entering the desired data at the data inputs. The output will change independent of the level of the clock input. This feature allows the counters to be used as modulo-N dividers by simply modifying the count length with the preset inputs.

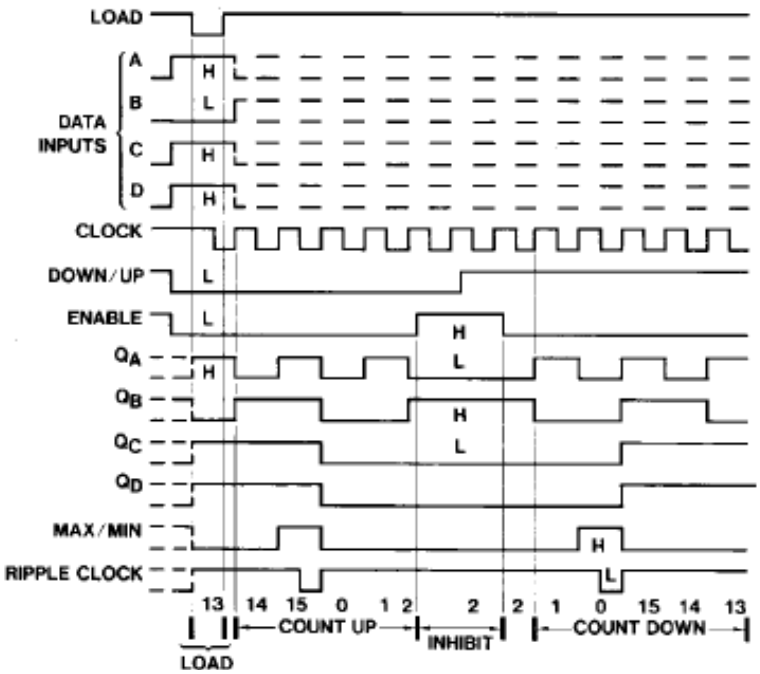
The clock, down/up, and load inputs are buffered to lower the drive requirement; which significantly reduces the number of clock drivers, etc., required for long parallel words.

Two outputs have been made available to perform the cascading function: ripple clock and maximum/minimum count. The latter output produces a high-level output pulse with duration approximately equal to one complete cycle of the clock when the counter overflows or underflows. The ripple clock output produces a low-level output pulse equal in width to the low-level portion of the clock input when an overflow or underflow condition exists. The counters can be easily cascaded by feeding the ripple clock output to the enable input of the succeeding counter if parallel clocking is used, or to the clock input if parallel enabling is used. The maximum/minimum count output can be used to accomplish look-ahead for high-speed operation.

Connection Diagram



Timing Diagram

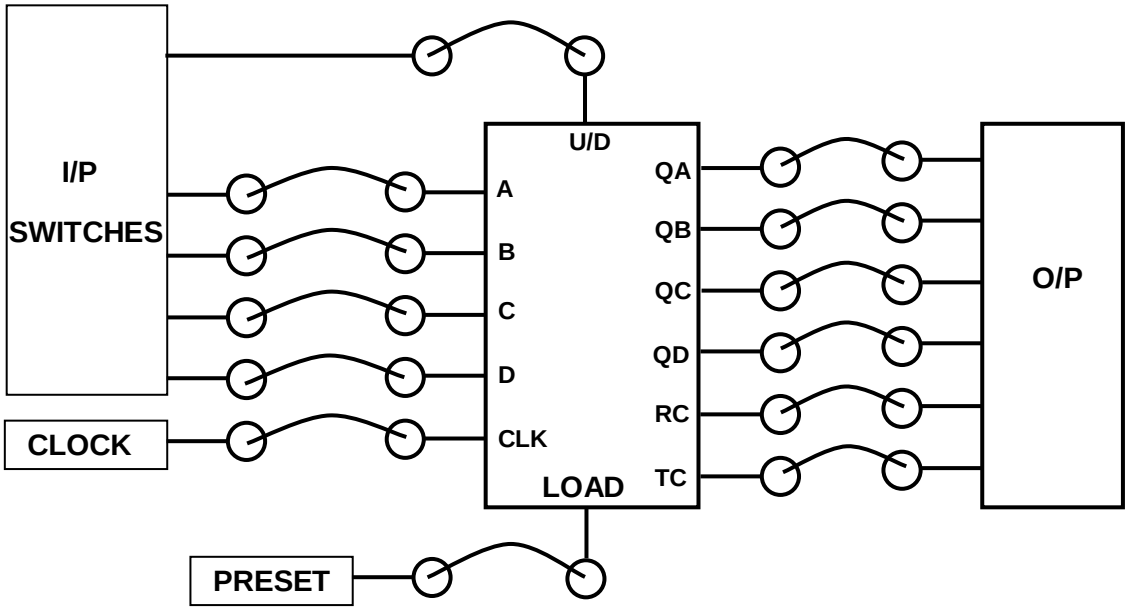


Functional table for 74191:

LOAD	ENABLE 'G'	DOWN/UP	CLOCK	FUNCTION
H	L	L	↑	Count up
H	L	H	↑	Count Down
L	X	X	X	Load
H	H	X	X	No change

PROCEDURE:-

- Do the connection as per block diagram shown. And switch on the power supply.



- Provide proper logic inputs from I/P Switched.
- To perform Count UP Operation initially set ABCD = 0000, U/D input to logic '0' and press PRESET switch once to load the data. As you load the data into counter led indicator will display the same data.
- Now provide clock to the counter manually by pressing the clock switch and observe the data on LEDs. It should follow the following table.

SR NO	CLOCK PULSES	OUTPUTS			
		QD	QC	QB	QA
1	1	0	0	0	1
2	2	0	0	1	0
3	3	0	0	1	1
4	4	0	1	0	0
5	5	0	1	0	1
6	6	0	1	1	0
7	7	0	1	1	1
8	8	1	0	0	0
9	9	1	0	0	1
10	10	1	0	1	0
11	11	1	0	1	1
12	12	1	1	0	0
13	13	1	1	0	1
14	14	1	1	1	0
15	15	1	1	1	1
16	16	0	0	0	0

- After 15th clock pulse TC (Max/Min) line goes high for one clock pulse duration. And for 16th clock pulse RC goes low for half duration of clock pulse & again goes high.

- For count down mode of counter load ABCD = 1111, U/D input to logic ‘1’ and press PRESET switch once to load the data in to counter. As you load the data into counter led indicator will display the same data i.e. 1111.
- Now provide clock to the counter manually by pressing the clock switch and observe the data on LEDs. It should follow the following table.

SR NO	CLOCK PULSES	OUTPUTS			
		QD	QC	QB	QA
1	1	1	1	1	0
2	2	1	1	0	1
3	3	1	1	0	0
4	4	1	0	1	1
5	5	1	0	1	0
6	6	1	0	0	1
7	7	1	0	0	0
8	8	0	1	1	1
9	9	0	1	1	0
10	10	0	1	0	1
11	11	0	1	0	0
12	12	0	0	1	1
13	13	0	0	1	0
14	14	0	0	0	1
15	15	0	0	0	0
16	16	1	1	1	1

- After 15th clock pulse TC (Max/Min) line goes high for one clock pulse duration. And for 16th clock pulse RC goes low for half duration of clock pulse & again goes high.

EXPERIMENT NO. 11

EXPERIMENT NO.11

NAME:-

Study of Decade Counter

OBJECTIVE:-

To study of synchronous binary counter using IC 7490.

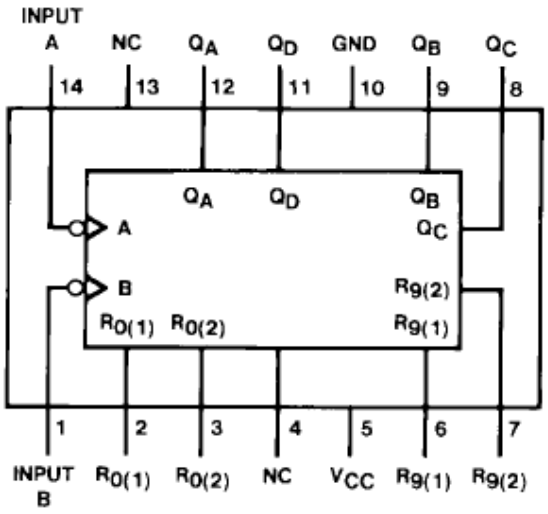
EQUIPMENTS:-

- BEL-DIT Board
- Power supply

THEORY:-

The 7490 monolithic counter contains four master slave flip-flops and additional gating to provide a divide-by two counter and a three-stage binary counter for which the count cycle length is divide-by-five.
The counter has a gated zero reset and also has gated set to nine inputs for used in BCD nine's complement applications.
To use the maximum count length (decade or four-bit binary), the B input is connected to the Q_A output. The input count pulses are applied to input A and the outputs are as described in the appropriate Function Table. A symmetrical divide-by-ten count can be obtained from the counters by connecting the Q_D output to the A input and applying the input count to the B input which gives a divide by-ten square wave at output Q_A.

Connection Diagram



1) BCD Count sequence when O/P Q_A is connected to input B for BCD count.

Count	Outputs			
	Q_D	Q_C	Q_B	Q_A
0	L	L	L	L
1	L	L	L	H
2	L	L	H	L
3	L	L	H	H
4	L	H	L	L
5	L	H	L	H
6	L	H	H	L
7	L	H	H	H
8	H	L	L	L
9	H	L	L	H

2) BCD Count sequence when O/P Q_D is connected to input A for Bi-quinary count.

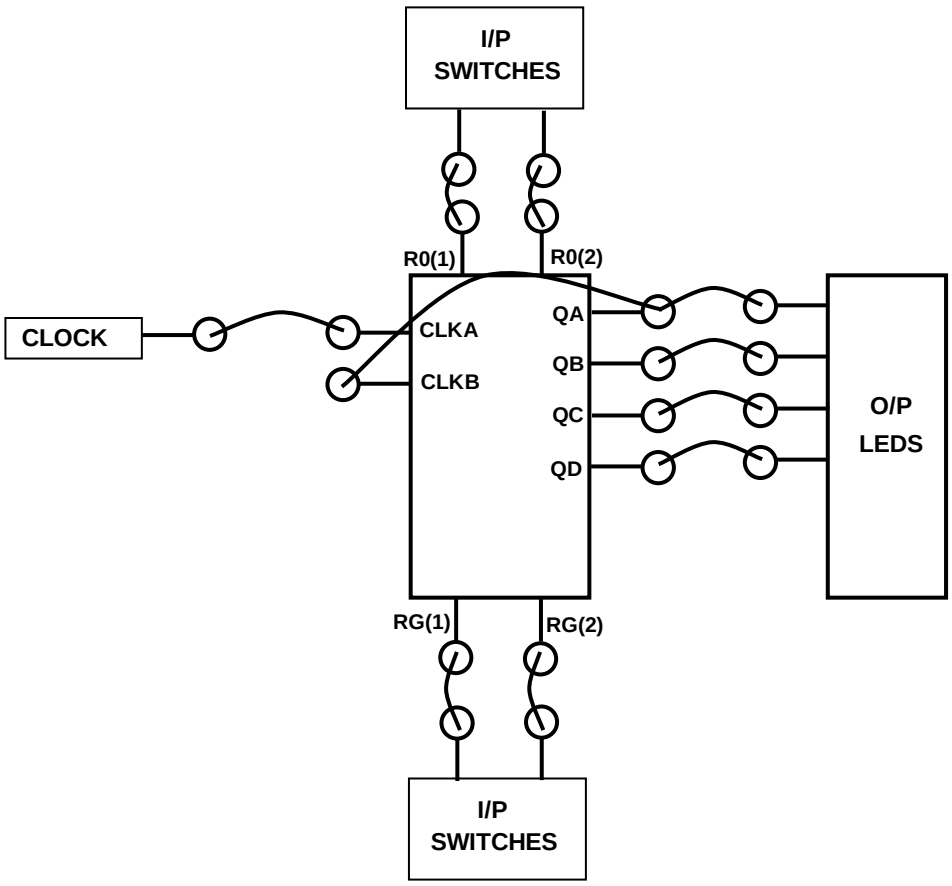
Count	Outputs			
	Q_A	Q_D	Q_C	Q_B
0	L	L	L	L
1	L	L	L	H
2	L	L	H	L
3	L	L	H	H
4	L	H	L	L
5	H	L	L	L
6	H	L	L	H
7	H	L	H	L
8	H	L	H	H
9	H	H	L	L

RESET/ COUNT function table

Reset Inputs				Outputs			
R0(1)	R0(2)	R9(1)	R9(2)	Q_D	Q_C	Q_B	Q_A
H	H	L	X	L	L	L	L
H	H	X	L	L	L	L	L
X	X	H	H	H	L	L	H
X	L	X	L	COUNT			
L	X	L	X	COUNT			
L	X	X	L	COUNT			
X	L	L	X	COUNT			

H = HIGH Level
L = LOW Level
X = Don't Care

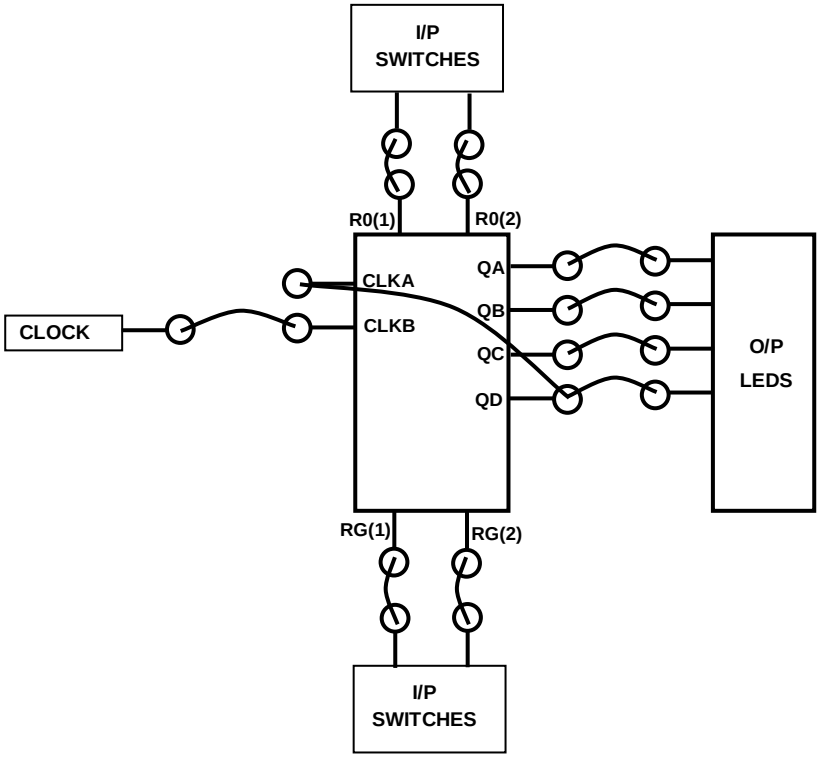
PROCEDURE:-



- Do the connection as shown in block diagram above and switch ON the power supply.
- Provide the proper logic inputs to R0 (1), R0 (2), RG (1) and RG (2) by referring its RESET/ COUNT function table
- Now provide Clock pulse one at a time by pressing Clock switch &observe the led indication at O/P section. It should be as shown in table.

SR NO	CLOCK PULSES	OUTPUTS			
		QD	QC	QB	QA
1	1	0	0	0	0
2	2	0	0	0	1
3	3	0	0	1	0
4	4	0	0	1	1
5	5	0	1	0	0
6	6	0	1	0	1
7	7	0	1	1	0
8	8	0	1	1	1
9	9	1	0	0	0
10	10	1	0	0	1

- Once the count reached to 1001 counter resets to 0000. That means it count 10 clock pulses and counter advances its counts by ten.
- The 7490 can be configured in following mode also.



- Do the connection as shown in block diagram above
- Provide the proper logic inputs to R0 (1), R0 (2), RG (1) and RG (2) by referring its RESET/ COUNT function table.
- Now provide Clock pulse one at a time by pressing Clock switch & observe the LED indication at O/P section. It should be as shown in table.

SR NO	CLOCK PULSES	OUTPUTS			
		QA	QD	QC	QB
1	1	0	0	0	1
2	2	0	0	1	0
3	3	0	0	1	1
4	4	0	1	0	0
5	5	1	0	0	0
6	6	1	0	0	1
7	7	1	0	1	0
8	8	1	0	1	1
9	9	1	1	0	0
10	10	0	0	0	0

- In this case counter output is not in sequence as in earlier case. Once the count reached to 1100 counter resets to 0000. That means it count 10 clock pulses.

EXPERIMENT NO. 12

EXPERIMENT NO.12

NAME:-

Study of Shift Register

OBJECTIVE:-

To study of Universal shift register using IC 74194.

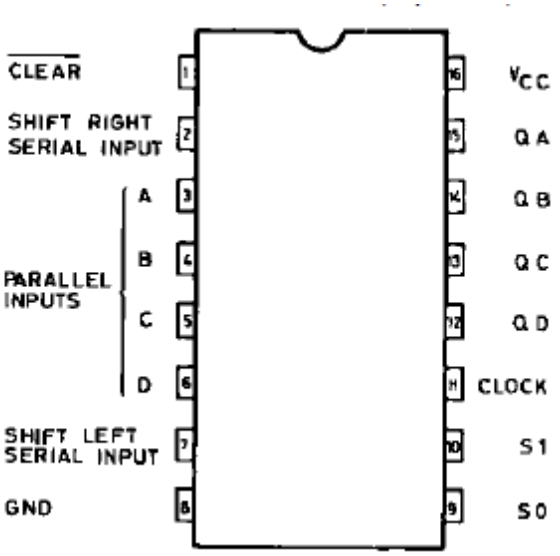
EQUIPMENTS:-

- BEL-DIT Board
- Power supply

THEORY:-

The 74194 is a high speed CMOS 4BIT PIPO SHIFT REGISTER fabricated in silicon gate C^2MOS technology. It has the same high speed performance of LSTTL combined with true CMOS low power consumption. This SHIFT REGISTER is designed to incorporate Virtually all of the features a system designer may want in a shift register. It features parallel inputs, parallel outputs, right shift and left shift serial inputs, clear line. The register has four distinct modes of operation: PARALLEL (broadside) LOAD; SHIFT RIGHT (in the direction QA QD); SHIFT LEFT; INHIBIT CLOCK (does nothing).
Synchronous parallel loading is accomplished by applying the four data bits and taking both mode control inputs, S0 and S1 high. The data are loaded into their respective flip-flops and appear at the outputs after the positive transition of the CLOCK input. During loading, serial data flow is inhibited.
Shift right is accomplished synchronously with the rising edge of the clock pulse when S0 is high and S1 is low. Serial data for this mode is entered at the SHIFT RIGHT data input. When S0 is low and S1 is high, data shifts left synchronously and new data is entered at the SHIFT LEFT serial input. Clocking of the flip-flops is inhibited when both mode control inputs are low. The mode control inputs should be changed only when the CLOCK input is high. All inputs are equipped with protection circuits against static discharge and transient excess voltage.

PIN DIAGRAM of 74194

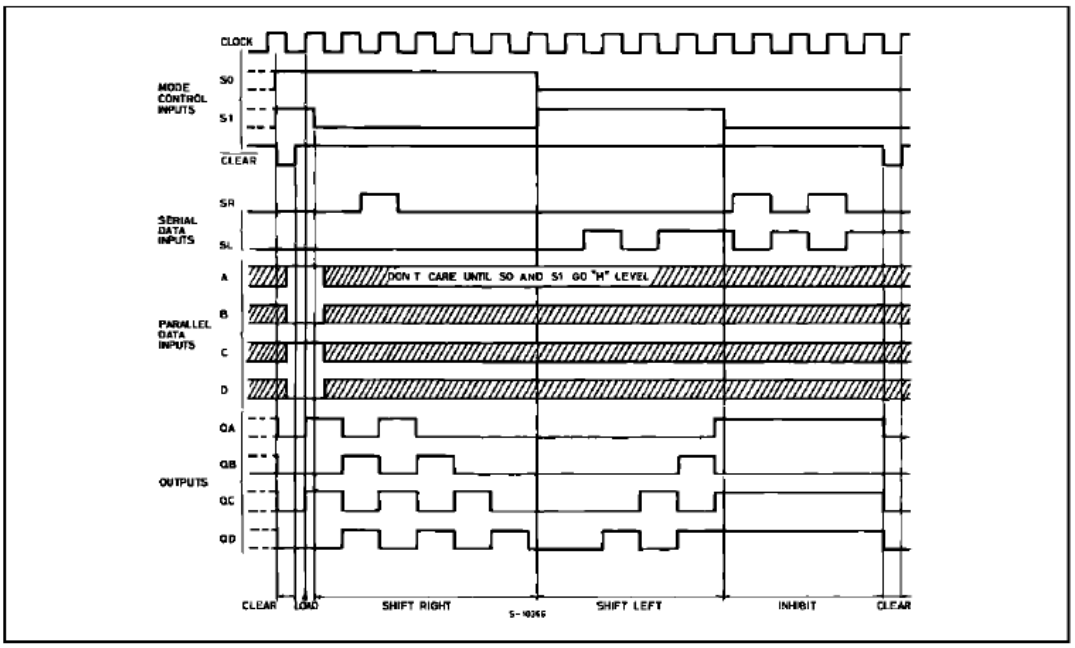


TRUTH TABLE

INPUTS										OUTPUS			
CLEAR	MODE		CLOCK	SERIAL		PARALLEL				QA	QB	QC	QD
	S1	S0		LEFT	RIGHT	A	B	C	D				
L	X	X	X	X	X	X	X	X	X	L	L	L	L
H	X	X	⊔	X	X	X	X	X	X	QA0	QB0	QC0	QD0
H	H	H	⊓	X	X	a	b	c	d	a	b	c	d
H	L	H	⊓	X	H	X	X	X	X	H	QAn	QBn	QCn
H	L	H	⊓	X	L	X	X	X	X	L	QAn	QBn	QCn
H	H	L	⊓	H	X	X	X	X	X	QBn	QCn	QDn	H
H	H	L	⊓	L	X	X	X	X	X	QBn	QCn	QDn	L
H	L	L	X	X	X	X	X	X	X	QA0	QB0	QC0	QD0

X: Don't Care : Don't Care
a ~ d : The level of steady state input voltage at input A ~ D respacfively
QA0 ~ QD0 : No change
QAn ~ QDn : The level of QA, QB, QC, respectively, before the mst recent positive transition of the clock.

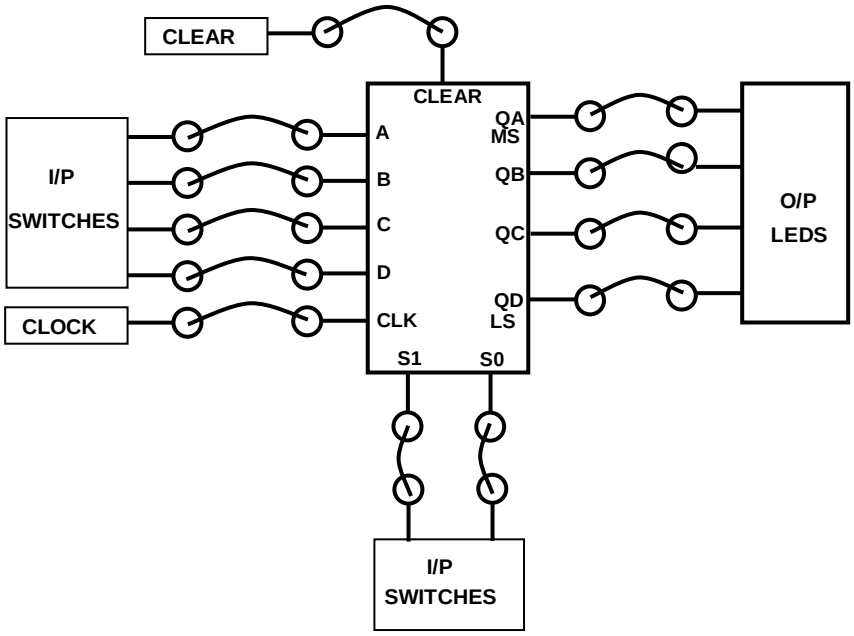
TIMING CHART



PROCEDURE:-

A) PIPO mode

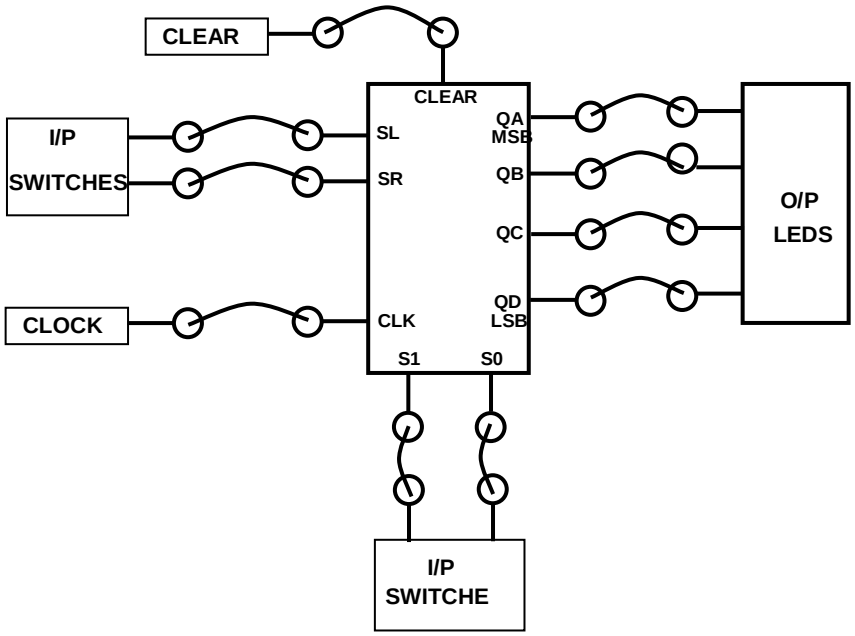
- Do the connection as shown in block diagram below.



- Set ABCD = 1010 using logic switches. Set S1 = S0 = '1' or Logic HIGH, connect Clear of Shift reg. to CLEAR terminal.
- Connect outputs Q_A to Q_D of reg. to LED indicators.
- Switch on the power supply. All Led indicators are in OFF positions.
- Now give clock signal to Shift register by CLOCK terminal, as soon as clock is reached to Reg. led indicators will show 1010, which is the input we have set for register.
- Now change the data at input side using I/P switches & press clock switch, LED indication now displays the new data. It means this shift register works as parallel in parallel out under clock signal control.

B) Right Shit Register

- Do the connection as per block diagram shown below,



- Set S1 = '0', S0 = '1', SL = X, SR = '1'. Connect Clear of Shift Reg. to CLEAR terminal.
- Connect outputs Q_A to Q_D of reg. to LED indicators.
- Switch on the power supply. All Led indicators are in OFF positions.
- Now give clock signal to Shift register by CLOCK terminal and observe the LED indication. The led indication should follow the sequence as shown in table.

SR	CLOCK	Q_A	Q_B	Q_C	Q_D	O/P
1	0	0	0	0	0	0
2	1	1	0	0	0	8
3	2	1	1	0	0	12
4	3	1	1	1	0	14
5	4	1	1	1	1	15

- From the above function table we can conclude that this register work as right shift register as it shifts '1' towards right by one position at every clock pulse.
- To start the counting again or to reset the register press CLEAR.

C) Left Shift Register

- Do the connection as per Right Shift register.

- Set S1= ‘1’, S0 = ‘0’, SL = ‘1’ and SR = X. Connect Clear of Shift Register to CLEAR terminal.
- Connect outputs QA to QD of reg. to LED indicators.
- Switch on the power supply. All Led indicators are in OFF positions.
- Now give clock signal to Shift register by CLOCK terminal and observe the LED indication. The led indication should follow the sequence as shown in table

SR NO	CLOCK	QA (MSB)	QB	Qc	QD	O/P in DEC.
1	0	0	0	0	0	0
2	1	0	0	0	1	1
3	2	0	0	1	1	3
4	3	0	1	1	1	7
5	4	1	1	1	1	15

- From the above function table we can conclude that this register work as Left shift register as it shifts ‘1’ towards left by one position at every clock pulse.
- To start the counting again or to reset the register press CLEAR.

EXPERIMENT NO. 13

EXPERIMENT NO.13

NAME:-

Study of Parity Checker

OBJECTIVE:-

To study of Even and Odd parity Checker using IC 74280

EQUIPMENTS:-

- BEL-DIT Board
- Power supply

THEORY:-

A parity bit is used for detecting errors during transmission of binary information. A parity bit is an extra bit included with a binary message to make the number is either even or odd. The message including the parity bit is transmitted and then checked at the receiver ends for errors. An error is detected if the checked parity bit doesn't correspond to the one transmitted. The circuit that generates the parity bit in the transmitter is called a 'parity Generator' and the circuit that checks the parity in the receiver is called a 'Parity checker'.
In even parity, the added parity bit will make the total number is even amount. In odd parity, the added parity bit will make the total number is odd amount. The parity checker circuit checks for possible errors in the transmission. If the information is passed in even parity, then the bits required must have an even number of 1's. An error occur during transmission, if the received bits have an odd number of 1's indicating that, one bit has changed in value during transmission.
This universal, monolithic, nine bit parity generator/checker utilize Schottky -clamped TTL high performance circuitry and feature ODD/ EVEN outputs to facilitate operation of either Odd or Even parity application. The word length capability is easily expanded by cascading.
74280 parity generator/checker offer the designer a trade-off between reduced power consumption and high performance. This device can be used to upgrade the performance of most systems utilizing the 74180 parity generator/checker.
This device is fully compatible with most other TTL circuits. All 74280 are buffered to lower the drive requirement.

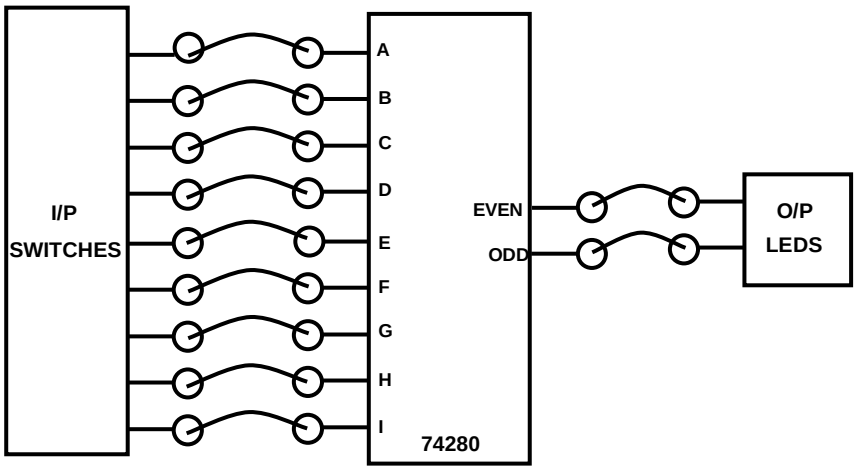
FUNCTION TABLE

NUMBER OF INPUTS A THRU I THAT ARE HIGH	OUTPUTS	
	Σ EVEN	Σ ODD
0, 2, 4, 6, 8	H	L
1, 3, 5, 7, 9	L	H

H = high level, L = low level

PROCEDURE:-

- Do the connections as per block diagram shown below.
- Apply logic inputs as shown in its function table and observe the corresponding parity indication
- For example set A to I = 101010101. In this data pattern No Of '1' (Logic High) are odd, thus ODD line must be high.
- Now set data A to I = 110011000. In this data pattern No Of '1' (Logic High) are Even, thus EVEN line must be high.
- In the similar way 74280 can be designed to generate the parity coded data.



EXPERIMENT NO. 14

EXPERIMENT NO.14

NAME:-

Study of Multiplexer / Demultiplexer

OBJECTIVE:-

To study of Multiplexer IC 74153 and Demultiplexer IC 74138

EQUIPMENTS:-

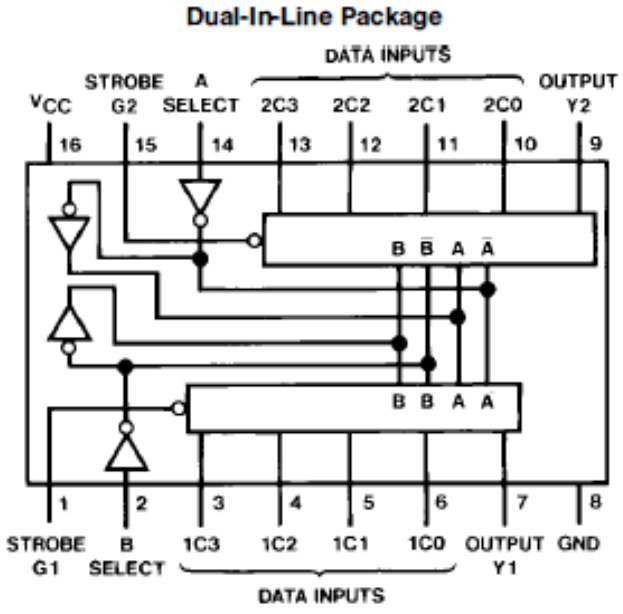
- BEL-DIT Board
- Power supply

THEORY:-

Multiplexer
Multiplexer means transmitting a large number of information units over a smaller number of channels or lines. A digital multiplexer is a combinational circuit that selects binary information from one of many input lines and directs it to a single output line. The selection of a particular input line is controlled by a set of selection lines. Normally there are 2^n input line and n selection lines whose bit combination determine which input is selected.

IC75153: Each of these data selectors/multiplexers contains inverters and drivers to supply fully complementary, on-chip, binary decoding data selection to the AND-OR-invert gates. Separate strobe inputs are provided for each of the two four-line sections.

Internal Block Diagram OF 74153:

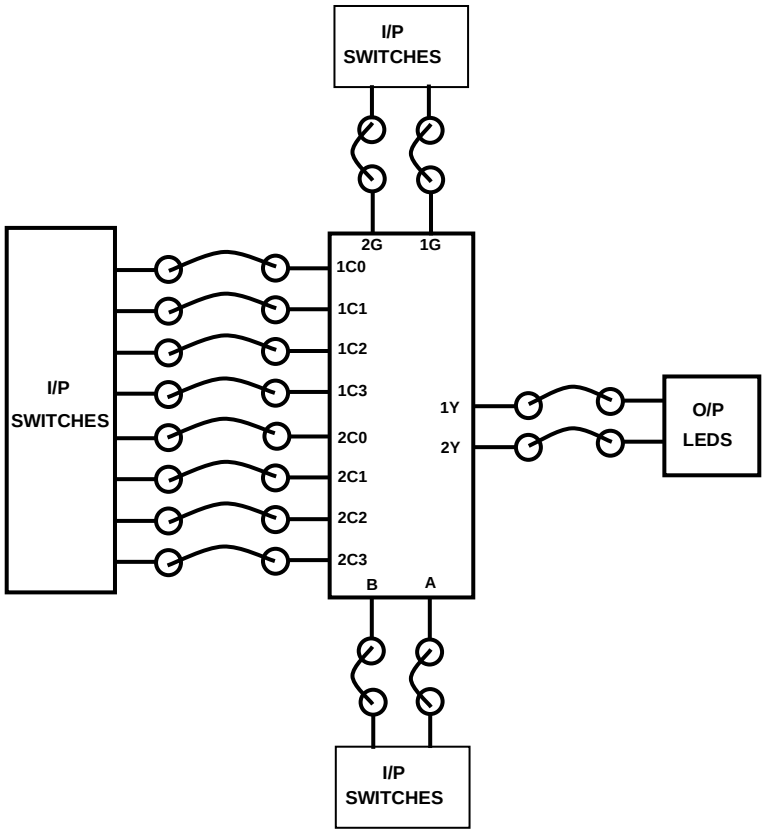


Function table of 74153

Select I/p		STROBE		DATA INPUTS				OUTPIT	
B	A	2G	1G	1C3	1C2	1C1	1C0	2Y	1Y
X	X	H	H	X	X	X	X	L	L
CASE 1									
L	L	H	L	L	L	L	**H**	L	**H**
L	H	H	L	L	L	**L**	H	L	**L**
H	L	H	L	L	**H**	L	L	L	**H**
H	H	H	L	**H**	L	L	L	L	**H**
CASE 2									
B	A	2G	1G	2C3	2C2	2C1	2C0	2Y	1Y
L	L	L	H	H	H	H	**L**	**L**	L
L	H	L	H	H	H	**H**	L	**H**	L
H	L	L	H	L	**H**	L	L	**H**	L
H	H	L	H	**L**	L	L	L	**L**	L

PROCEDURE:-

- Do the connection as per block diagram shown below and switch ON the power supply.
- Apply proper logic inputs o the Multiplexer and observe the output on LEDs.
- Verify the function table of multiplexer in both the cases.



Application of multiplexer as full Adder and full Subtractor

Full Adder

Function table of Full Adder is as shown below;

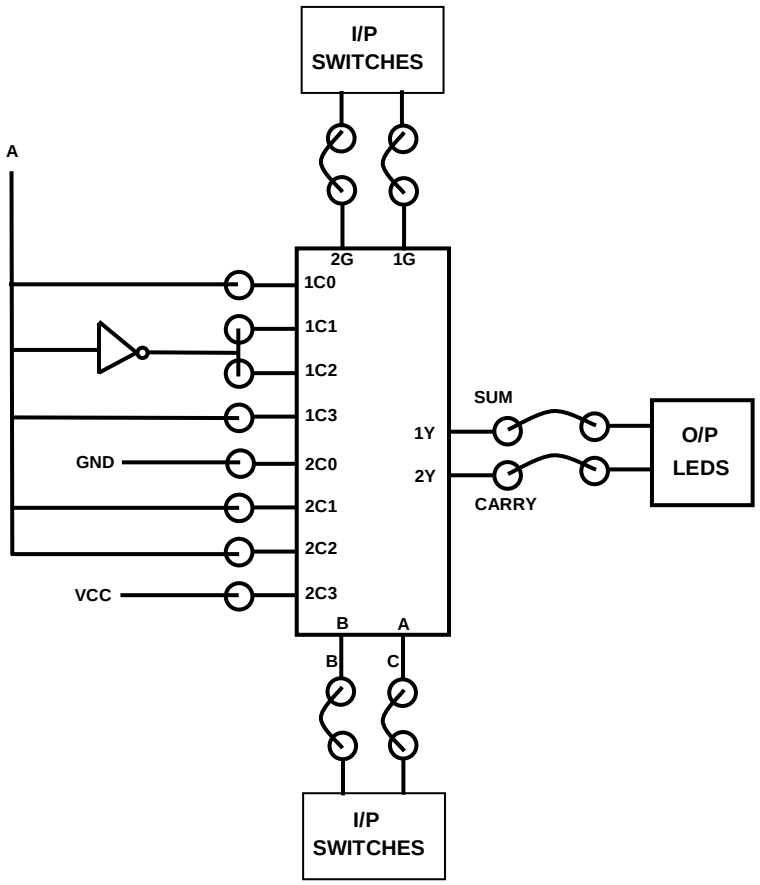
INPUTS			OUTPUTS	
A	B	C	SUM	CARRY
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

74153 is dual 4:1 multiplexer. It has 2 select lines and it has two o/p lines. We can design Full adder using 74153 multiplexer as shown below;

	C0	C1	C2	C3
SUM / 1Y	A	A!	A!	A
CARRY / 2Y	0	A	A	1

From function table of full adder LSB B&C connect to select inputs of multiplexer B and A respectively & MSB A to the inputs of multiplexer as shown in above table. The output 1Y will function as SUM output & 2Y will work as carry output. The connection diagram for Full adder using Multiplexer 74153 is as shown below. Connect both strobe lines 1g & 2g to logic low or '0'. Now apply inputs A, B & C as ADDER inputs to Multiplexer and observe the output at multiplexer out (1Y & 2Y) which is functioning as Full Adder. Verify all possible combinations of full adder using its function table.

MULTIPLEXER AS FULL ADDER



Full Subtractor

Function table of Full Subtractor:

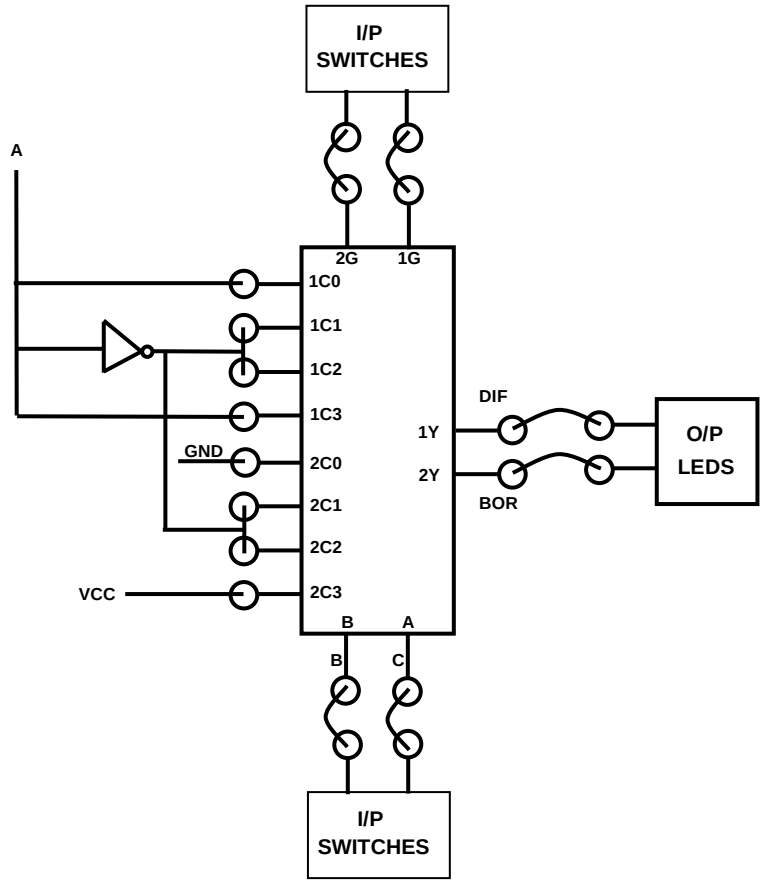
INPUTS			OUTPUTS	
A	B	C	DIFFERENCE	BORROW
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

	C0	C1	C2	C3
DIFF / 1Y	A	A!	A!	A
BOR / 2Y	0	A!	A!	1

Full Subtractor Design

Do the connection for Full Subtractor as shown in block diagram below. Apply inputs A, B & C as SUBTRACTOR inputs to Multiplexer and observe the output at multiplexer out (1Y & 2Y) which is functioning as Full Subtractor. Verify all possible combinations of full subtractor using its function table.

Multiplexer as Full Subtractor

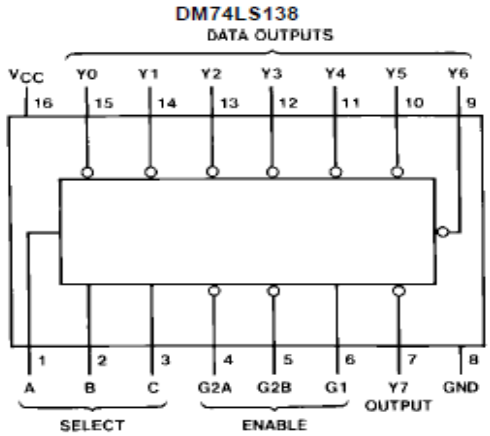


Demultiplexer

The function of Demultiplexer is in contrast to multiplexer function. It takes information from one line and distributes it to a given number of output lines. For this reason, the demultiplexer is also known as a data distributor. Decoder can also be used as demultiplexer.
In the 1: 4 demultiplexer circuit, the data input line goes to all of the AND +9gates. The data select lines enable only one gate at a time and the data on the data input line will pass through the selected gate to the associated data output line.

The 74138 decodes one-of-eight lines, based upon the condition at the three binary select inputs and the three enable inputs. Two active-low and one active-high enable inputs reduce the need for external gates or inverters when expanding. A 24-line decoder can be implemented with no external inverters, and a 32-line decoder requires only one inverter. An enable input can be used as a data input for demultiplexing applications.
All of these decoders/Demultiplexer feature fully buffered inputs, presenting only one normalized load to its driving circuit. All inputs are clamped with high-performance Schottky diodes to suppress line-ringing and simplify system design.

Connection Diagram for 74138:



Function table for 74138:

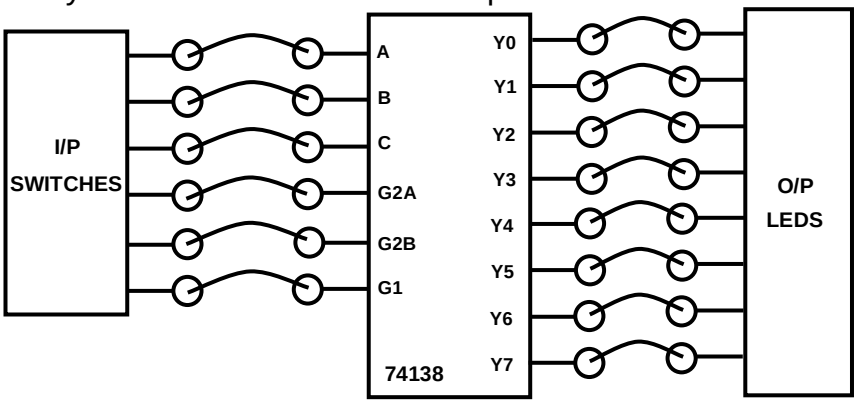
DM74LS138

Inputs					Outputs							
Enable		Select										
G1	G2 (Note 1)	C	B	A	YO	Y1	Y2	Y3	Y4	Y5	Y6	Y7
X	H	X	X	X	H	H	H	H	H	H	H	H
L	X	X	X	X	H	H	H	H	H	H	H	H
H	L	L	L	L	L	H	H	H	H	H	H	H
H	L	L	L	H	H	L	H	H	H	H	H	H
H	L	L	H	L	H	H	L	H	H	H	H	H
H	L	L	H	H	H	H	H	L	H	H	H	H
H	L	H	L	L	H	H	H	H	L	H	H	H
H	L	H	L	H	H	H	H	H	H	L	H	H
H	L	H	H	L	H	H	H	H	H	H	L	H
H	L	H	H	H	H	H	H	H	H	H	H	L

Note 1: G2 = G2A and G2B

PROCEDURE:-

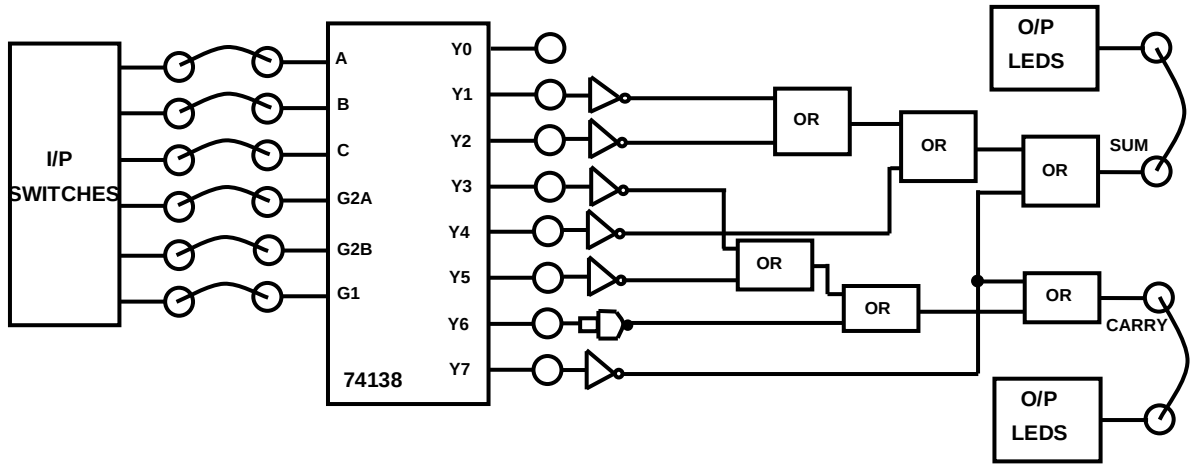
- Do the connection as per block diagram shown below and switch ON the power supply.
- Apply proper logic inputs o the Demultiplexer and observe the output on LEDs.
- Verify the function table of Demultiplexer.



4) Application of Demultiplexer as full Adder and full Subtractor.

Full Adder

Refer the function table of Full adder shown above. To design full adder simply or the outputs of Demultiplexer for which output is high. The connection diagram for Demultiplexer as full adder is as shown below,

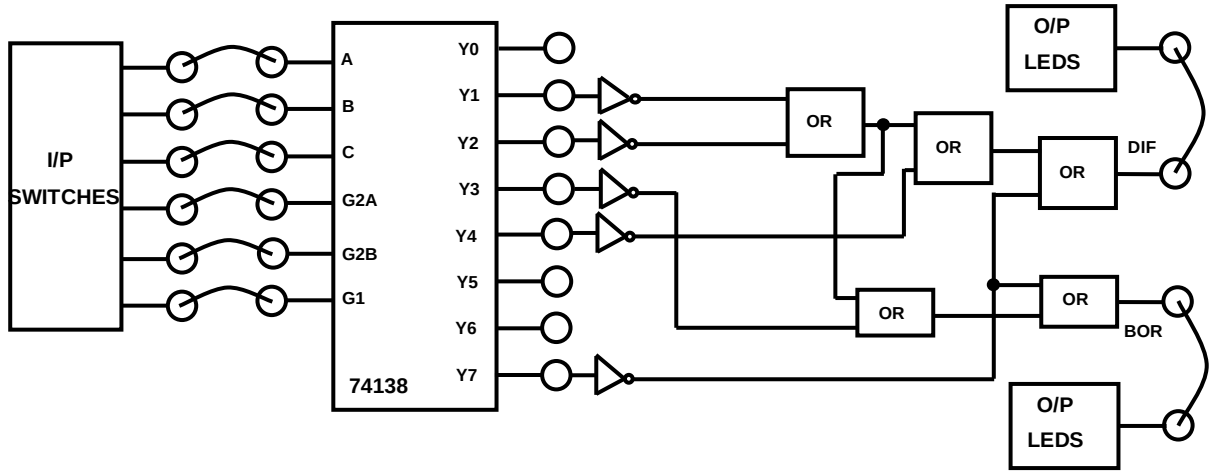


As 74138 have active low outputs, it is necessary to add the inverters at the outputs. As from the function table of full adder there are Total 7 inverters required and in BEL-DIT Boards there are 6 inverter available. Thus 7th inverter is built using NAND gate by shorting its both inputs (NAND as NOT).

Ones the connections are over then apply inputs to Demultiplexer and observe its final output. Verify the functionality of Full Adder using Demultiplexer.

5) Application of Demultiplexer as full Adder and full Subtractor.

FULL SUBTRACTOR: Refer the function table of Full subtractor shown above. To design full subtractor simply or the outputs of Demultiplexer for which output is high The connection diagram for Demultiplexer as full subtractor is as shown below,



Ones the connections are over then apply inputs to Demultiplexer and observe its final output. Verify the functionality of Full Subtractor using Demultiplexer.

EXPERIMENT NO. 15

EXPERIMENT NO.15

NAME:-

Study of Seven Segment Decoder

OBJECTIVE:-

To study of seven segment decoder driver IC 7447.

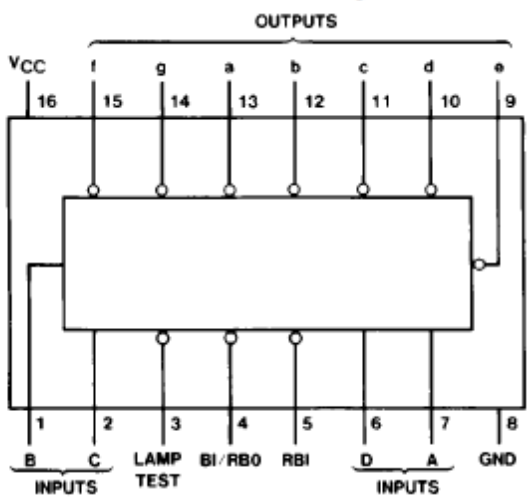
EQUIPMENTS:-

- BEL-DIT Board
- Power supply

THEORY:-

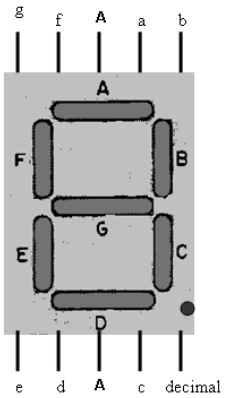
The 7447 feature active-low outputs designed for driving common-anode LEDs or incandescent indicators directly. All of the circuits have full ripple-blanking input/output controls and a lamp test input. Segment identification and resultant displays are shown on a following page. Display patterns for BCD input counts above nine are unique symbols to authenticate input condition.
All of the circuits incorporate automatic leading and/or trailing-edge, zero-blanking control (RBI and RBO). Lamp test (LT) of these devices may be performed at any time when the BI/RBO node is at a high logic level. All types contain an overriding blanking input (BI) which can be used to control the lamp intensity (by pulsing) or to inhibit the outputs.

Connection Diagram



7 Segment Display

The 7 Segment Display consists of 7 LEDs which can be glowed in some particular fashion using the 7 different input pins. For example, if a, b, g, e, d glow, numeral 2 is represented A stands for Anode, i.e. A has to be connected to 5 V. The decimal is optional as it is used to glow the decimal point in bottom right of the 7 segment display.



Function table for 7447

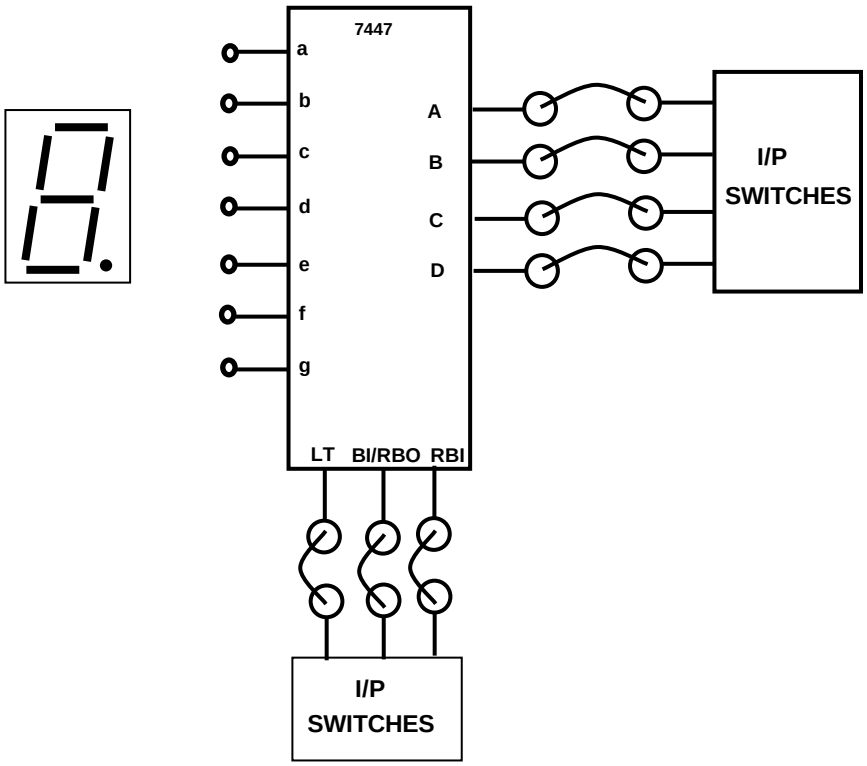
Decimal or Function	Inputs						BI/RBO (Note 1)	Outputs							
	LT	RBI	D	C	B	A		a	b	c	d	e	f	g	
0	H	H	L	L	L	L	H	L	L	L	L	L	L	H	
1	H	X	L	L	L	H	H	H	L	L	H	H	H	H	
2	H	X	L	L	H	L	H	L	L	H	L	L	H	L	
3	H	X	L	L	H	H	H	L	L	L	L	H	H	L	
4	H	X	L	H	L	L	H	H	L	L	H	H	L	L	
5	H	X	L	H	L	H	H	L	H	L	L	H	L	L	
6	H	X	L	H	H	L	H	H	H	L	L	L	L	L	
7	H	X	L	H	H	H	H	L	L	L	H	H	H	H	
8	H	X	H	L	L	L	H	L	L	L	L	L	L	L	
9	H	X	H	L	L	H	H	L	L	L	H	H	L	L	
10	H	X	H	L	H	L	H	H	H	H	L	L	H	L	
11	H	X	H	L	H	H	H	H	H	L	L	H	H	L	
12	H	X	H	H	L	L	H	H	L	H	H	H	L	L	
13	H	X	H	H	L	H	H	L	H	H	L	H	L	L	
14	H	X	H	H	H	L	H	H	H	H	L	L	L	L	
15	H	X	H	H	H	H	H	H	H	H	H	H	H	H	
BI	X	X	X	X	X	X	L	H	H	H	H	H	H	H	NOTE 3
RBI	H	L	L	L	L	L	L	H	H	H	H	H	H	H	NOTE 4
LT	L	X	X	X	X	X	H	L	L	L	L	L	L	L	NOTE 5

NOTE 2

Note 1: BI/RBO is a wire-AND logic serving as blanking input (BI) and/or ripple-blanking output (RBO).
Note 2: The blanking input (BI) must be open or held at a high logic level when output functions 0 through 15 are desired. The ripple-blanking input (RBI) must be open or high if blanking of a decimal zero is not desired.
Note 3: When a low logic level is applied directly to the blanking input (BI), all segment outputs are high regardless of the level of any other input.
Note 4: When ripple-blanking input (RBI) and inputs A, B, C, and D are at a low level with the lamp test input high, all segment outputs go H and the ripple blanking output (RBO) goes to a low level (response condition).
Note 5: When the blanking input/ripple-blanking output (BI/RBO) is open or held high and a low is applied to the lamp-test input, all segment outputs are L.
H = High level, L = Low level, X = Don't Care

PROCEDURE:-

- Do the connection as per block diagram shown below and switch on the power supply.
- For normal operation set LT = ‘1’ RBI = ‘1’ and BI/RBO = ‘1’. Apply input to the IC from I/P switches as per the function table and observe the output on seven segment display
- You can give output of onboard Decade counter (7490) as input to seven segment decoder. Observe the output on Display. It displays from 0 to 9 digits.
- You can also give output of onboard Binary counter (74191) as input to seven segment decoder. Observe the output on Display. It displays digits as shown in the function table for 0 to 15.



EXPERIMENT NO. 16

EXPERIMENT NO.16

NAME:-

Study of Comparator

OBJECTIVE:-

To study of 4 Bit Comparator IC 7485

EQUIPMENTS:-

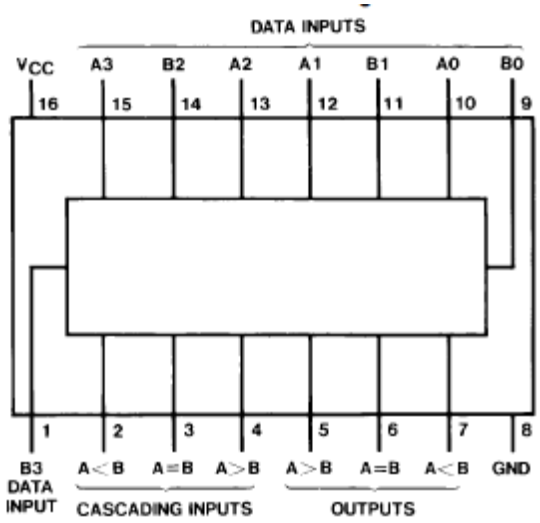
- BEL-DIT Board
- Power supply

THEORY:-

The comparison of two numbers is an operator that determines one number is greater than, less than (or) equal to the other number. A magnitude comparator is a combinational circuit that compares two numbers A and Band determines their relative magnitude. The outcome of the comparator is specified by three binary variables that indicate whether A>B, A=B (or) A<B.

These 4-bit magnitude comparators perform comparison of straight binary or BCD codes. Three fully-decoded decisions about two, 4-bit words (A, B) are made and are externally available at three outputs. These devices are fully expandable to any number of bits without external gates. Words of greater length may be compared by connecting comparators in cascade. The A l B, A k B, and A e B outputs of a stage handling less-significant bits are connected to the corresponding inputs of the next stage handling more-significant bits. The stage handling the least-significant bits must have a high-level voltage applied to the A e B input. The cascading path is implemented with only a two-gate-level delay to reduce overall comparison times for long words.

Pin Diagram of 7485



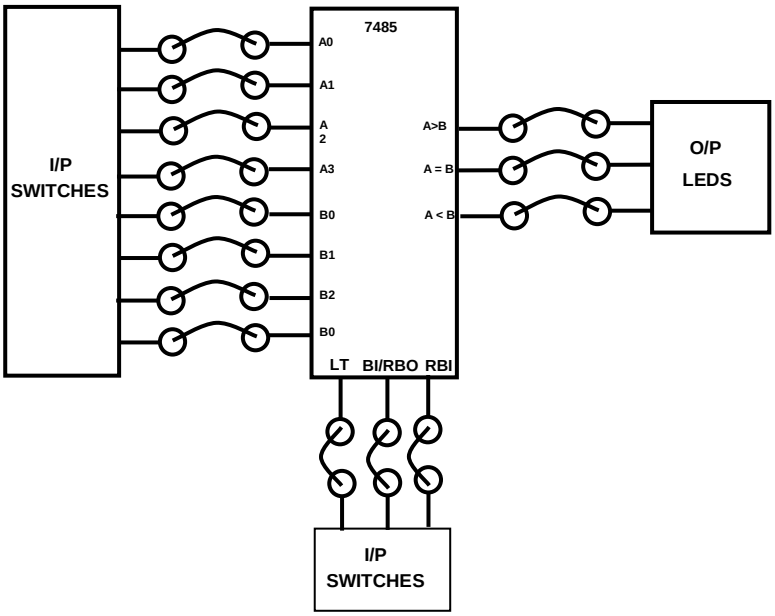
FUNCTION TABLE FOR 7485

Comparing Inputs				Cascading Inputs			Outputs		
A3, B3	A2, B2	A1, B1	A0, B0	A > B	A < B	A = B	A > B	A < B	A = B
A3 > B3	X	X	X	X	X	X	H	L	L
A3 < B3	X	X	X	X	X	X	L	H	L
A3 = B3	A2 > B2	X	X	X	X	X	H	L	L
A3 = B3	A2 < B2	X	X	X	X	X	L	H	L
A3 = B3	A2 = B2	A1 > B1	X	X	X	X	H	L	L
A3 = B3	A2 = B2	A1 < B1	X	X	X	X	L	H	L
A3 = B3	A2 = B2	A1 = B1	A0 > B0	X	X	X	H	L	L
A3 = B3	A2 = B2	A1 = B1	A0 < B0	X	X	X	L	H	L
A3 = B3	A2 = B2	A1 = B1	A0 = B0	H	L	L	H	L	L
A3 = B3	A2 = B2	A1 = B1	A0 = B0	L	H	L	L	H	L
A3 = B3	A2 = B2	A1 = B1	A0 = B0	L	L	H	L	L	H
A3 = B3	A2 = B2	A1 = B1	A0 = B0	X	X	H	L	L	H
A3 = B3	A2 = B2	A1 = B1	A0 = B0	H	H	L	L	L	L
A3 = B3	A2 = B2	A1 = B1	A0 = B0	L	L	L	H	H	L

H = High Level, L = Low Level, X = Don't Care

PROCEDURE:-

- Do the connection as per block diagram shown below and switch ON the power supply.
- Give step by step inputs to A & B of comparator starting from MSB (A3 and B3).
- Initially just observe the comparison between inputs A & B inputs and ignore the cascading inputs.
- Once all possible combinations for A & B inputs are over then apply cascading inputs as per function table. Observe the outputs of comparator and verify it with function table.
- Cascading inputs are used to increase the input line capacity of comparator.



EXPERIMENT NO. 17

EXPERIMENT NO.17

NAME:-

Study of Code Converters

OBJECTIVE:-

To study of Binary to Gray, Gray to Binary, Binary to BCD, BCD to Binary, BCD to Excess 3 and Excess 3 to BCD converters

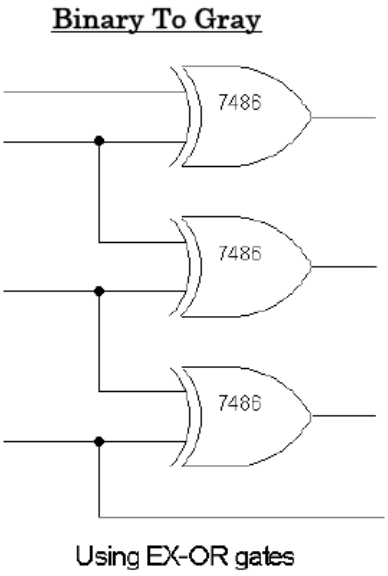
EQUIPMENTS:-

- BEL-DIT Board
- Power supply

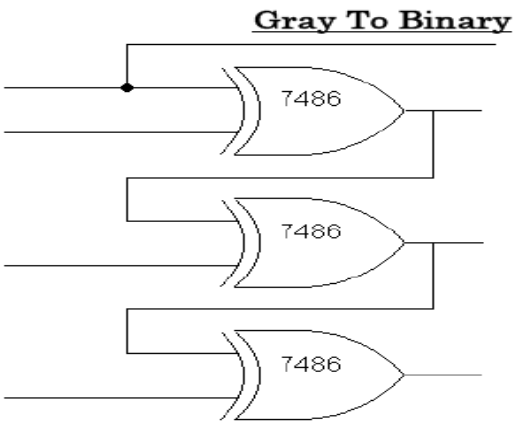
THEORY:-

A) Binary to Gray and Gray to Binary Conversion

Logic diagram of Binary to Gray code conversion



Logic diagram of Gray to Binary code conversion

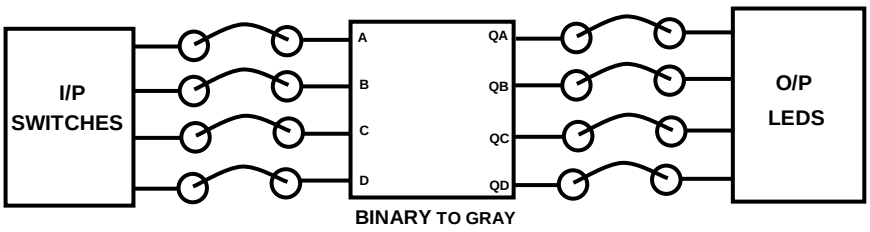


Function table / Truth table for Binary to gray and vice versa.

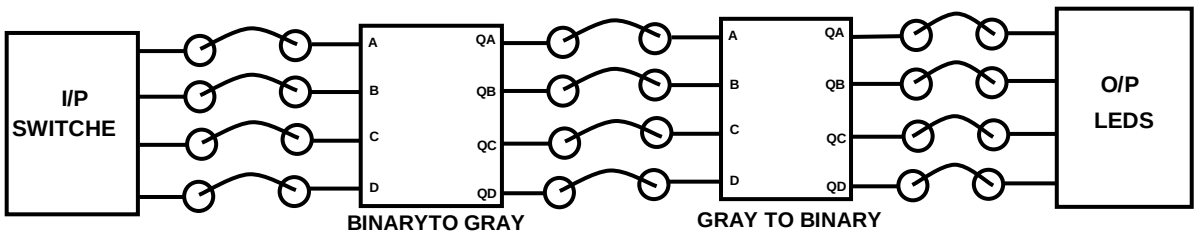
Inputs				Outputs			
B3	B2	B1	B0	G3	G2	G1	G0
0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	1
0	0	1	0	0	0	1	1
0	0	1	1	0	0	1	0
0	1	0	0	0	1	1	0
0	1	0	1	0	1	1	1
0	1	1	0	0	1	0	1
0	1	1	1	0	1	0	0
1	0	0	0	1	1	0	0
1	0	0	1	1	1	0	1
1	0	1	0	1	1	1	1
1	0	1	1	1	1	1	0
1	1	0	0	1	0	1	0
1	1	0	1	1	0	1	1
1	1	1	0	1	0	0	1
1	1	1	1	1	0	0	0

PROCEDURE:-

- Do the connections as per block diagram shown below and switch on the power supply.



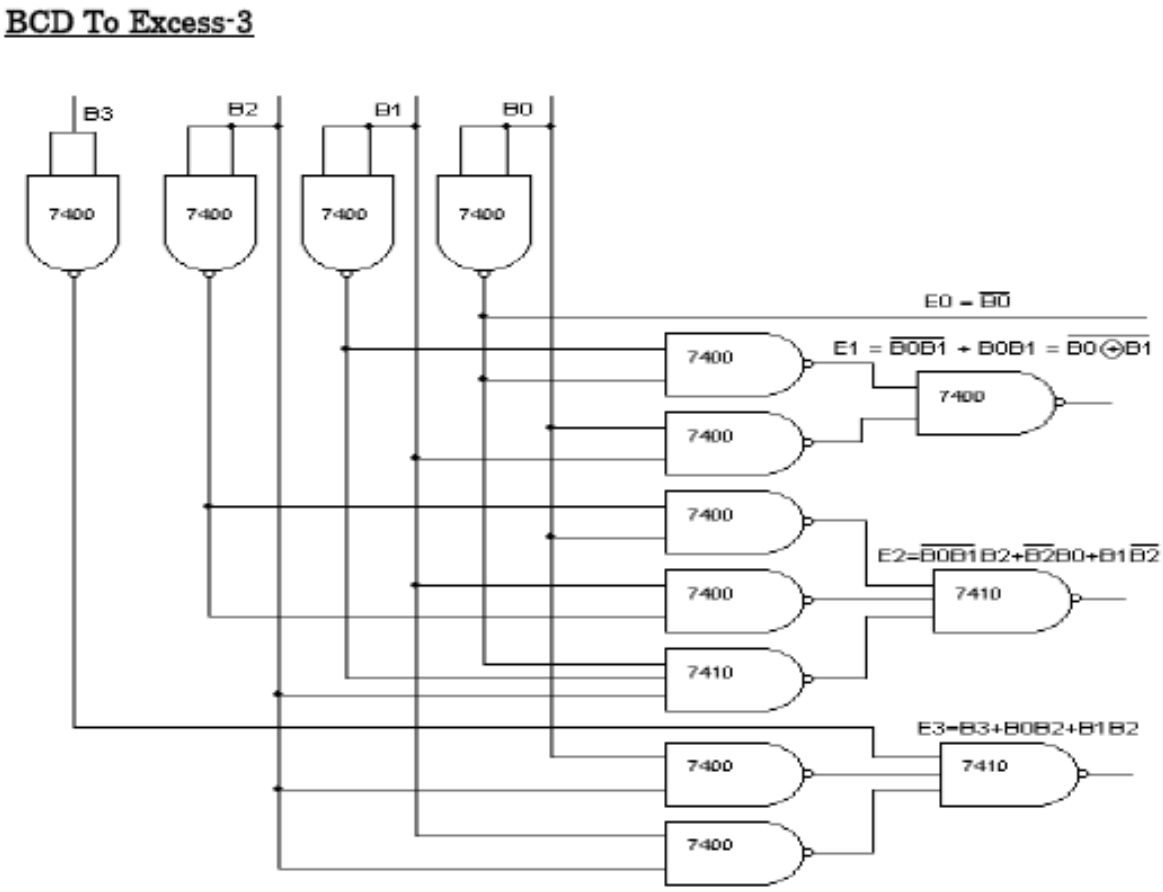
- Apply logic inputs to the block diagram from I/P switches and observe the corresponding generated code on LEDs at O/P section.
- Verify the truth table for binary to gray code conversion.
- For Gray to Binary do the connection as shown below,



- Apply logic inputs to the block diagram from I/P switches and observe the corresponding generated code on LEDs at O/P section.
- Verify the truth table for Gray to Binary code conversion.

B) BCD to EXCESS-3 and EXCESS-3 to BCD Conversion

Logic diagram of BCD to EXCESS-3 using NAND gates

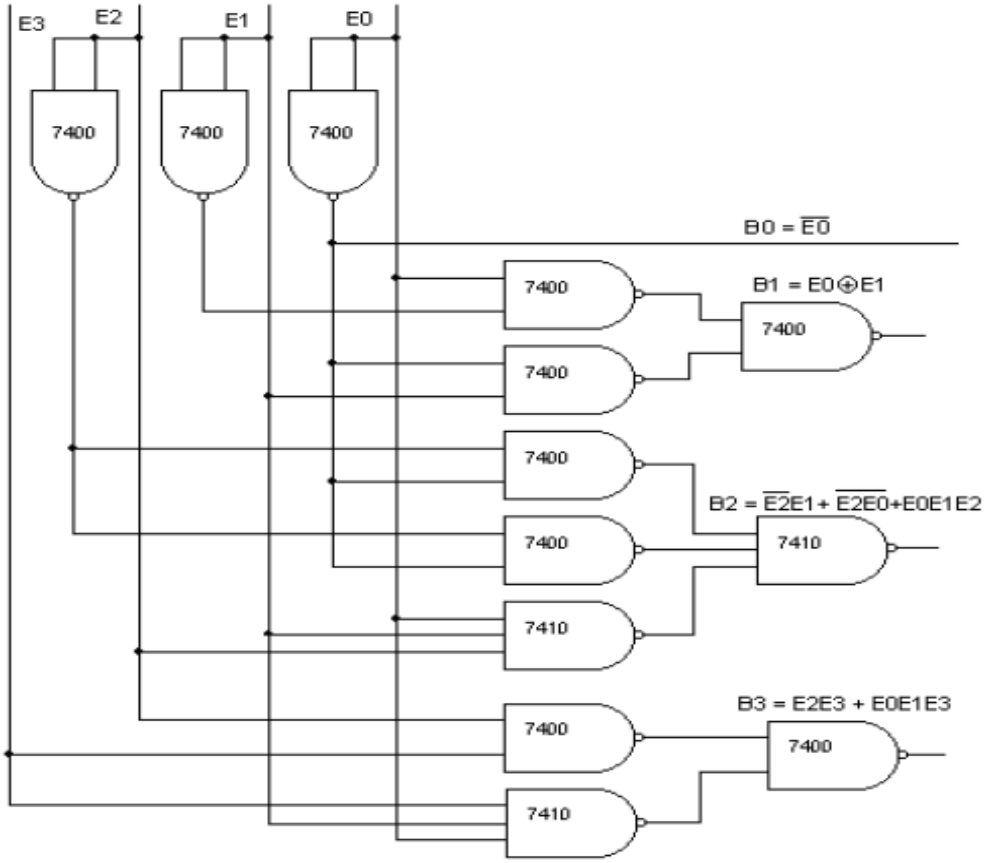


Truth Table for BCD to EXCESS-3 code conversion

Inputs				Outputs			
D	C	B	A	QD	QC	QB	QA
0	0	0	0	0	0	1	1
0	0	0	1	0	1	0	0
0	0	1	0	0	1	0	1
0	0	1	1	0	1	1	0
0	1	0	0	0	1	1	1
0	1	0	1	1	0	0	0
0	1	1	0	1	0	0	1
0	1	1	1	1	0	1	0
1	0	0	0	1	0	1	1
1	0	0	1	1	1	0	0

Logic diagram of EXCESS-3 to BCD using NAND gates

Excess-3 To BCD :-

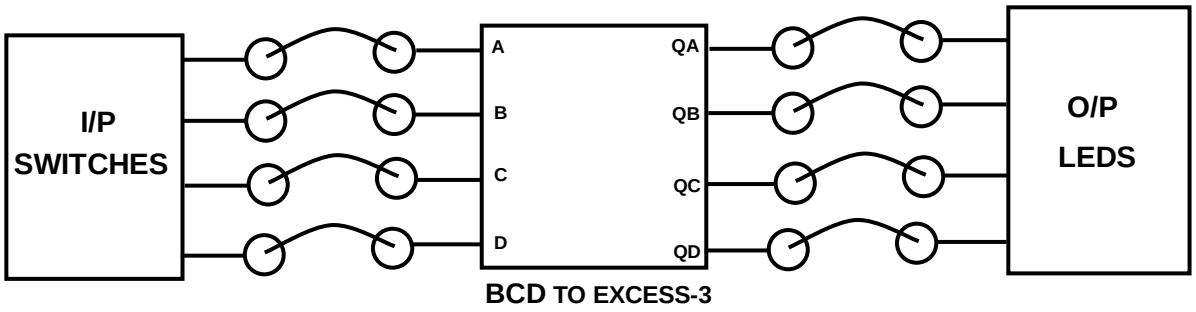


Truth Table for EXCESS-3 to BCD code conversion

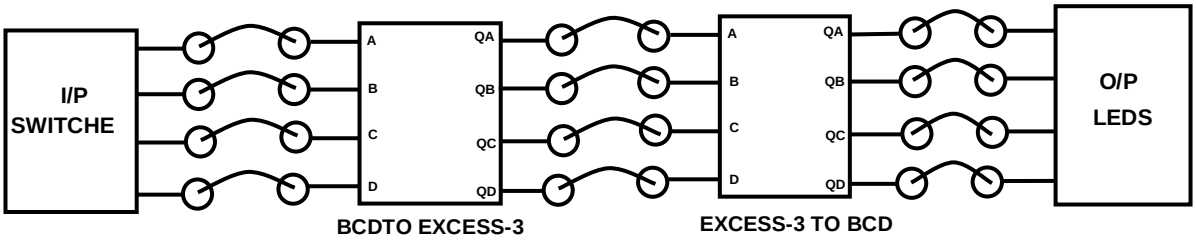
Inputs				Outputs			
D	C	B	A	QD	QC	QB	QA
0	0	1	1	0	0	0	0
0	1	0	0	0	0	0	1
0	1	0	1	0	0	1	0
0	1	1	0	0	0	1	1
0	1	1	1	0	1	0	0
1	0	0	0	0	1	0	1
1	0	0	1	0	1	1	0
1	0	1	0	0	1	1	1
1	0	1	1	1	0	0	0
1	1	0	0	1	0	0	1

PROCEDURE:-

- Do the connections as per block diagram shown below and switch on the power Supply.



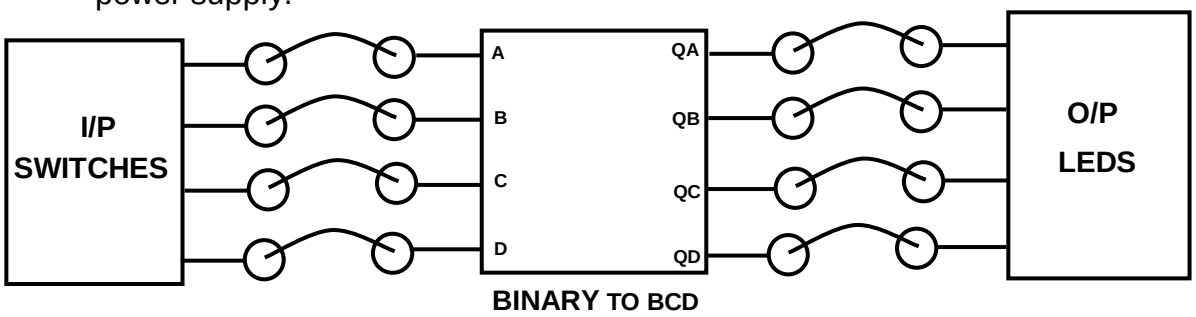
- Apply logic inputs to the block diagram from I/P switches and observe the Corresponding generated code on LEDs at O/P section.
- Verify the truth table for BCD to EXCESS-3 code conversion.
- For EXCESS-3 to BCD do the connection as shown below,



- Apply logic inputs to the block diagram from I/P switches and observe the corresponding generated code on LEDs at O/P section.
- Verify the truth table for EXCESS-3 to BCD code conversion.

C) Binary to BCD and BCD to Binary Conversion

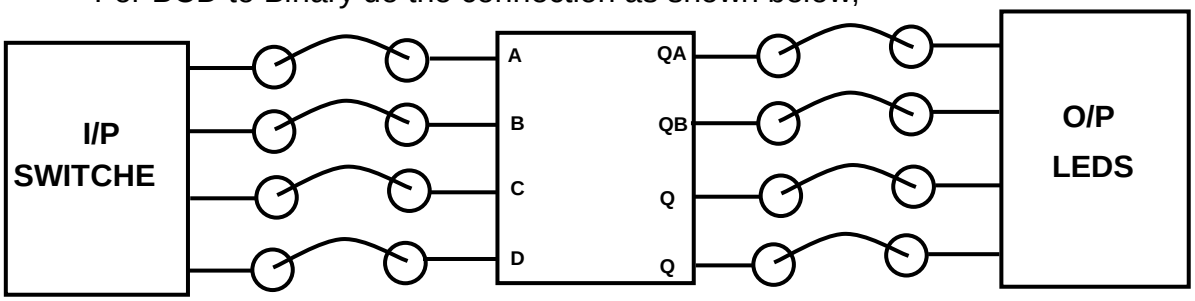
- Do the connections as per block diagram shown below and switch on the power supply.



- Apply logic inputs to the block diagram from I/P switches and observe the corresponding generated code on LEDs at O/P section.
- Verify the truth table for Binary to BCD code conversion as shown below.

DECIMAL NUMBER	BINARY INPUT				BCD OUTPUT			
	D	C	B	A	QD	QC	QB	QA
0	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**
1	**0**	**0**	**0**	**1**	**0**	**0**	**0**	**1**
2	**0**	**0**	**1**	**0**	**0**	**0**	**1**	**0**
3	**0**	**0**	**1**	**1**	**0**	**0**	**1**	**1**
4	**0**	**1**	**0**	**0**	**0**	**1**	**0**	**0**
5	**0**	**1**	**0**	**1**	**0**	**1**	**0**	**1**
6	**0**	**1**	**1**	**0**	**0**	**1**	**1**	**0**
7	**0**	**1**	**1**	**1**	**0**	**1**	**1**	**1**
8	**1**	**0**	**0**	**0**	**1**	**0**	**0**	**0**
9	**1**	**0**	**0**	**1**	**1**	**0**	**0**	**1**
10	**1**	**0**	**1**	**0**	**x**	**x**	**x**	**x**
11	**1**	**0**	**1**	**1**	**x**	**x**	**x**	**x**
12	**1**	**1**	**0**	**0**	**x**	**x**	**x**	**x**
13	**1**	**1**	**0**	**1**	**x**	**x**	**x**	**x**
14	**1**	**1**	**1**	**0**	**x**	**x**	**x**	**x**
15	**1**	**1**	**1**	**1**	**x**	**x**	**x**	**x**

- For BCD to Binary do the connection as shown below,



BCDTO BINARY

- Apply logic inputs to the block diagram from I/P switches and observe the corresponding generated code on LEDs at O/P section.
- Verify the truth table for BCD to Binary code conversion as shown below.
-

DECIMAL NUMBER	BCD INPUT				BINARY OUTPUT			
	D	C	B	A	QD	QC	QB	QA
0	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**
1	**0**	**0**	**0**	**1**	**0**	**0**	**0**	**1**
2	**0**	**0**	**1**	**0**	**0**	**0**	**1**	**0**
3	**0**	**0**	**1**	**1**	**0**	**0**	**1**	**1**
4	**0**	**1**	**0**	**0**	**0**	**1**	**0**	**0**
5	**0**	**1**	**0**	**1**	**0**	**1**	**0**	**1**
6	**0**	**1**	**1**	**0**	**0**	**1**	**1**	**0**
7	**0**	**1**	**1**	**1**	**0**	**1**	**1**	**1**
8	**1**	**0**	**0**	**0**	**1**	**0**	**0**	**0**
9	**1**	**0**	**0**	**1**	**1**	**0**	**0**	**1**
10	**1**	**0**	**1**	**0**	**x**	**x**	**x**	**x**
11	**1**	**0**	**1**	**1**	**x**	**x**	**x**	**x**
12	**1**	**1**	**0**	**0**	**x**	**x**	**x**	**x**
13	**1**	**1**	**0**	**1**	**x**	**x**	**x**	**x**
14	**1**	**1**	**1**	**0**	**x**	**x**	**x**	**x**
15	**1**	**1**	**1**	**1**	**x**	**x**	**x**	**x**

EXPERIMENT NO. 18

EXPERIMENT NO.18

NAME:-

Study of 4 Bit Binary Adder

OBJECTIVE:-

To study four bit Binary adder with fast carry using IC 7483.

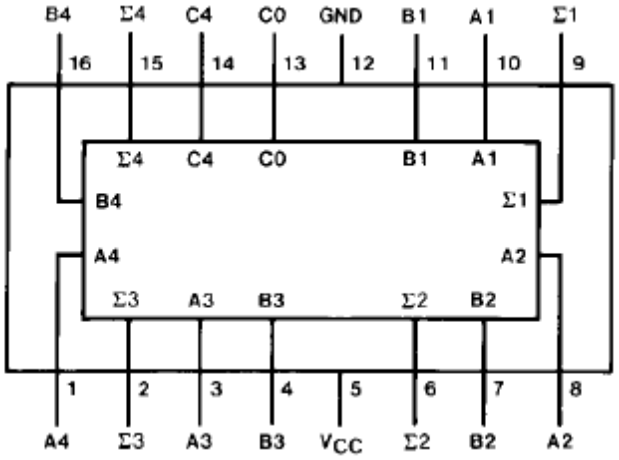
EQUIPMENTS:-

- BEL-DIT board
- Power supply
- IC7483

THEORY:-

These full adders perform the Additional of two 4-bit binary numbers. The sum (Σ) outputs are provided for each bit and the resultant carry (C4) is obtained from the fourth bit. These adders feature full internal look ahead across all four bits. This provides the system designer with partial look ahead performance at the economy and reduced package count of a ripple-carry implementation.
The adder logic, including the carry, is implemented in its true form meaning that the end-around carry can be accomplished without the need for logic or level inversion.

Pin Diagram of 7483.



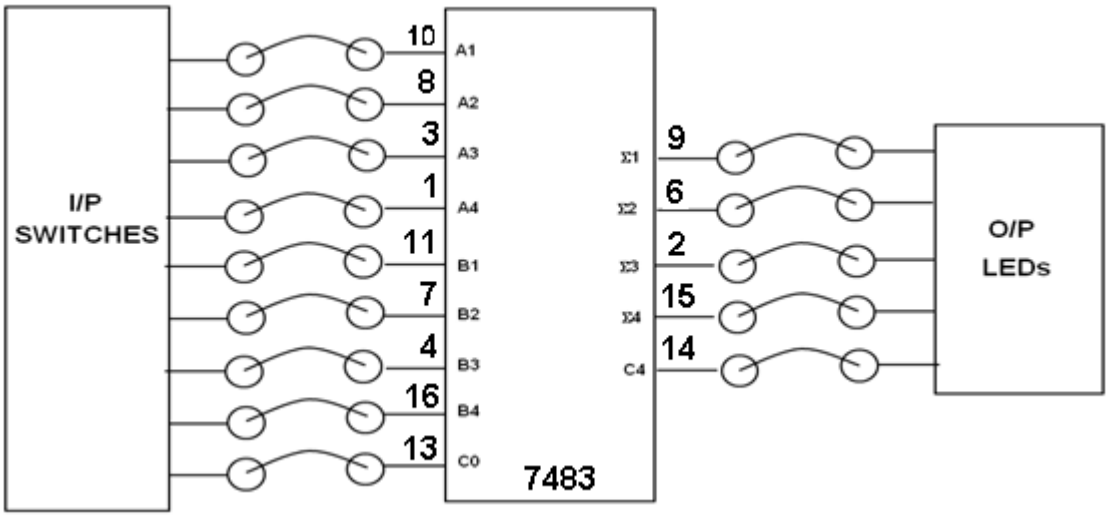
Truth table for 7483

Inputs				Outputs					
				When C0 = L			When C0 = H		
				When C2 = L			When C2 = H		
A1 / A3	B1 / B3	A2 / A4	B2 / B4	Σ1 / Σ3	Σ2 / Σ4	C2 / C4	Σ1 / Σ3	Σ2 / Σ4	C2 / C4
L	L	L	L	L	L	L	H	L	L
H	L	L	L	H	L	L	L	H	L
L	H	L	L	H	L	L	L	H	L
H	H	L	L	L	H	L	H	H	L
L	L	H	L	L	H	L	H	H	L
H	L	H	L	H	H	L	L	L	H
L	H	H	L	H	H	L	L	L	H
H	H	H	L	L	L	H	H	L	H
L	L	L	H	L	H	L	H	H	L
H	L	L	H	H	H	L	L	L	H
L	H	L	H	H	H	L	L	L	H
H	H	L	H	L	L	H	H	L	H
L	L	H	H	L	L	H	H	L	H
H	L	H	H	H	L	H	L	H	H
L	H	H	H	H	L	H	L	H	H
H	H	H	H	L	H	H	H	H	H

H= High Level, L= Low Level
Input condition at A1, B1, A2, B2 and C0 are used to determine outputs Σ1 and Σ2 and the value of the initial carry C2.
The values at C2, A3, B3, A4 and B4 are then used to determine outputs Σ1, Σ2 and C4.

PROCEDURE:-

- Mount IC 7483 on 20 pin ZIF socket. Connect +5V supply to Pin 5 and GND to Pin 12 of IC7483. Do the connections as per block diagram shown below and switch on the power supply.



- Apply logic inputs as per the block diagram from I/P switches and observe the sum and carry outputs on LED.
- Verify the truth table for all possible combinations.